Implementation and Verification of a CAN/CAN FD Protocol Controller in VHDL

Mads Richardt (s224948)

May 18, 2026

Abstract

This thesis describes the design, partial implementation, and verification of a CAN/CAN FD protocol controller in VHDL-93 for Everllence's FPGA-based IO-extender platform. The controller complies with ISO 11898-1 and supports CB, CE, FB, and FE frame formats with dual bit rate switching and Transmitter Delay Compensation for the FD data phase. The design follows the ISO 11898-1 layered reference model, with the MAC, PCS, and FCE sub-layers implemented as independently testable modules and connected to the host via Avalon-ST interfaces.

37 requirements were derived from ISO 11898-1, each linked to its source clause, verification method, and testbench assertion. 28 are closed against passing testbenches or code inspection. The nine open requirements comprise six LLC-layer requirements deferred pending LLC sub-layer implementation, one not applicable to this architecture, one optional operational feature, and one requiring a frame-aware reference model for error-type-specific injection coverage.

The design was synthesized on a Cyclone 10 LP FPGA target using 4,608 logic elements at 30% device utilization, ~4× the existing CAN Classic controller. Analysis attributes the majority of this growth to RX frame buffer decode logic, with protocol FSM logic growing 2.4× to 3.0× over the CC equivalent. The worst-case fmax of approximately 127 MHz exceeds the highest recommended CAN FD system clock of 80 MHz by more than a factor of 1.5.

Approval

This thesis was prepared at the Department of Applied Mathematics and Computer Science (DTU Compute), Technical University of Denmark, in partial fulfilment of the requirements for the degree of Bachelor of Science in Engineering. The work was carried out in collaboration with Everllence, where the controller is intended for use in high-reliability engine controller applications.

It is assumed that the reader has a working knowledge of digital logic design and binary communication protocols. Familiarity with VHDL or a similar hardware description language is an advantage but is not strictly required to follow the architectural and verification discussions.

Mads Richardt (s224948)

Kgs. Lyngby, May 2026

Acknowledgements

Edward Alexandru Todirica, Associate Professor, DTU Compute. Thank you for supervision throughout the project and for valuable guidance and feedback.

Fredrik Kristensen, Everllence. Thank you for guidance throughout the project, code and report review, and feedback on the design and implementation.

Alex Fihl Hedegaard Nielsen, Everllence. Thank you for the sparring and advice, good company - and the many coffee machine trips that kept the work moving.

Contents

Abbreviations

Table 1: Abbreviations used in this report.

Abbreviation	Meaning
ACK	Acknowledgment
AD	ACK Delimiter
AEF	Active Error Flag
AI	Artificial Intelligence
BRS	Bit Rate Switch
CAN	Controller Area Network
CANH	CAN High bus wire
CANL	CAN Low bus wire
CB	Classic Base (frame format)
CBFF	Classic Base Frame Format
CC	CAN Classic
CD	CRC Delimiter
CE	Classic Extended (frame format)
CEFF	Classic Extended Frame Format
CRC	Cyclic Redundancy Check
D	Dominant
DLC	Data Length Code
DMA	Direct Memory Access
DUT	Device Under Test
ED	Error Delimiter
EOF	End of Frame
ESI	Error State Indicator
FB	FD Base (frame format)
FBFF	FD Base Frame Format
FCE	Fault Confinement Entity
FD	Flexible Data Rate
FDF	FD Frame bit
FE	FD Extended (frame format)
FEFF	FD Extended Frame Format
FPGA	Field-Programmable Gate Array
FSB	Fixed Stuff Bit
FSM	Finite State Machine
FTYP	Frame Type
HDL	Hardware Description Language
IDB	Identifier Base field (11-bit base ID)
IDE	Identifier Extension bit
IEXT	Identifier Extension field (18-bit extended ID)
IFS	Inter Frame Space
INT	Intermission
IP	Intellectual Property

Abbreviation	Meaning
IPT	Information Processing Time
ISO	International Organization for Standardization
LE	Logic Element
LLC	Logical Link Control
LLM	Large Language Model
MAC	Medium Access Control
MIT	Massachusetts Institute of Technology (license)
OD	Overload Delimiter
OF	Overload Flag
OSVVM	Open Source VHDL Verification Methodology
PCS	Physical Coding Sublayer
PE	Phase Error
PEF	Passive Error Flag
PS	Propagation Segment (PROP_SEG)
PS1	Phase Segment 1 (PHASE_SEG1)
PS2	Phase Segment 2 (PHASE_SEG2)
PVT	Process, Voltage, Temperature
R	Recessive
RF	Remote Frame
RRS	Reserved Remote Request Substitution bit (FD frames)
RTL	Register Transfer Level
RTR	Remote Transmission Request
RX	Receiver / Receive
SB	Stuff Bit
SBC	Stuff Bit Count
SCP	Standard Corporate Protocol
SJW	Synchronization Jump Width
SOF	Start of Frame
SP	Sample Point
SRR	Substitute Remote Request
SS	Sync Segment (SYNC_SEG)
SSP	Secondary Sample Point
ST	Suspend Transmission
TDC	Transmitter Delay Compensation
TEC/REC	Transmit Error Counter / Receive Error Counter
TOML	Tom's Obvious Minimal Language
TQ	Time Quantum
TX	Transmitter / Transmit
VAN	Vehicle Area Network

List of Figures

Reading Guide

The report covers the full design and verification of a CAN FD protocol controller, from requirements extraction through RTL implementation to synthesis results. The sections below summarize each part to orient the reader before the detailed treatment begins.

- **Section 1** : Motivates the project in its industrial context at Everllence, introduces CAN Classic and CAN FD at the level of motivation and key properties through the background subsection (Section 1.1), surveys available CAN FD IP alternatives and explains why none satisfies the combined requirements, and states the three thesis objectives.
- **Section 2** : Describes how 37 requirements were derived from ISO 11898-1 using an AI-assisted extraction pipeline, and introduces the custom tooling developed to maintain them through iterative refinement.
- **Section 3** : Presents the key CAN protocol elements at the detailed mechanistic level. Covers the ISO sub-layer reference model, frame formats, bit timing, bit stuffing, CRC generation, and error detection and fault confinement. Readers familiar with ISO 11898-1 may skip this section.
- **Section 4** : Documents how each requirement is classified by sub-layer ownership, TX/RX scope, frame format applicability, observability, and verification method, and explains how these classifications drive the module decomposition and testbench architecture.
- **Section 5** : Presents the architectural design and the decisions that drove it. The key design elements follow directly from the verification plan established in Section 4.
- **Section 6** : Describes the RTL behavior of each module: `can_mac_fsm`, `can_mac_ser`, `can_mac_bs`, `can_mac_crc`, `can_fce`, and `can_pcs`.
- **Section 7** : Reports testbench execution outcomes for the five unit testbenches and the integration testbench, with waveform excerpts illustrating key protocol behaviors.
- **Section 8** : Reports resource utilization and timing results from synthesis on the Cyclone 10 LP target.
- **Section 9** : Interprets the logic element growth over the CAN Classic baseline implementation, assesses the three thesis objectives against the verification results (Section 9.1), and identifies future work (Section 9.2).
- **Section 10** : Summarizes the findings.
- **Appendices** : The signal-level schematic of `can_mac_pcs_fce` (Section A), and the full verification plan tables (Section B).

1 Introduction

Everllence (formerly MAN Energy Solutions) is a provider of propulsion and decarbonization solutions for the marine and energy industries, designing large two-stroke and four-stroke combustion engines for ship propulsion and power generation. This thesis was written within the engine controller division, where FPGA-based control hardware is developed and maintained for integration into Everllence's engine platforms. Industrial control systems for large marine engines demand communication protocols that combine fault tolerance and confinement, multi-master arbitration, and multi-decade service reliability. The Controller Area Network (CAN) communication protocol, developed by Bosch in the 1980s for automotive and industrial control, meets these basic demands [1].

The original version of CAN protocol, CAN Classic (CC), forms the basis of Everllence's existing CAN infrastructure, which is implemented through a VHDL CC protocol controller deployed on an IO-extender board forming part of the Triton motor controller system. As control systems grow more data-intensive, the eight-byte payload and 1 Mbit/s ceiling of CC has become a practical constraint. The CAN FD (Flexible Data Rate) protocol extension, introduced in 2012 [2], extends the maximum data payload to 64 bytes and raises the bit rate beyond 8 Mbit/s while preserving the arbitration and fault-confinement/tolerance architecture of CC. Thus, CAN FD (CF) is the natural migration path for Everllence's existing CAN infrastructure.

1.1 Background

This section establishes the technical context for the thesis work. CC and CF are both governed by ISO 11898-1 [3], which defines the CAN data link layer protocol. The following subsections CC and CF each at the level of motivation and key properties, and introduce the existing Everllence CC controller (`can_bus_controller`) that provided the immediate starting point for the redesign.

1.1.1 CAN Classic

CC implements a serial multi-master bus connecting electronic control units in automotive environments. CAN uses a shared two-wire differential bus on which all nodes broadcast simultaneously and arbitrate access without any designated bus master. Any node may initiate a transmission at any time. Contention is resolved by a non-destructive bitwise arbitration in which the transmitter with the lower-priority identifier detects the collision and silently withdraws, leaving the winner's frame intact. The bus uses two conductors, CANH and CANL, whose voltage difference encodes the bit value, see Figure 1. Twisting the conductors ensures that external electromagnetic interference couples equally onto both wires. Because the receiver measures only the differential voltage, this common-mode noise cancels - a practical necessity in the electrically harsh environment.

CC's error-handling architecture is a distinguishing feature relative to simpler serial protocols. Five complementary error detection mechanisms operate concurrently on every transmitted frame: transmitter bit monitoring, frame format checking, cyclic redundancy checking, acknowledgment checking, and bit stuffing violation detection. The residual error probability, the probability that a corrupted frame passes all detection mechanisms undetected, for an eight-byte frame in a ten-node network is on the order of 10^{-9} per frame - several orders of magnitude lower than contemporary automotive bus alternatives such as VAN and SCP [4]. A fault confinement mechanism tracks each node's error history and automatically disconnects persistently faulty nodes from the bus without disrupting communication between healthy nodes. Together these properties made CC the protocol of choice for safety-relevant in-vehicle networks.

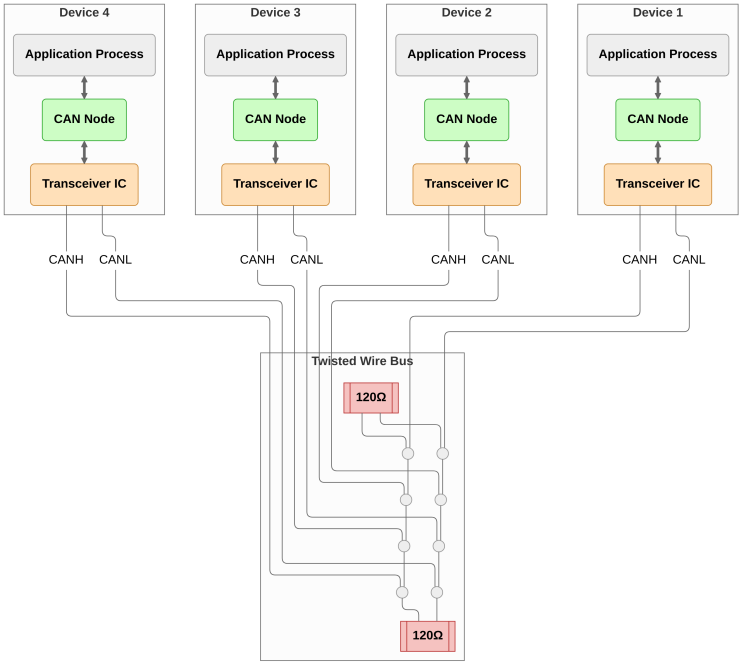

Figure 1: Four CAN nodes connected to a shared twisted-wire bus via CANH and CANL conductors. Each node comprises an application process, a CAN controller, and a transceiver IC. The bus is terminated with 120Ω resistors at each end.

1.1.2 CAN FD

The design goal of CF was to extend payload and bit rate while preserving the arbitration and fault-confinement architecture that distinguished CC from its contemporaries. CC's original data payload was capped at eight bytes per frame, limiting raw throughput to around 1 Mbit/s. As embedded control applications became more data-intensive, this ceiling became a practical constraint. CF extends the maximum payload to 64 bytes. In addition, it introduces a dual bit rate frame format comprising two phases: a higher-speed data phase with bit rates of 8 Mbit/s or beyond, and a nominal arbitration phase operating at CC bit-rates.

At high data phase bit rates the transceiver's TX-to-RX loop delay (approximately 100 ns for a typical CF transceiver such as the TCAN1042 [5]) may exceed the data phase bit time. Since the fault confinement mechanism relies on transmitter nodes monitoring their own transmitted bits, this necessitates the Transmitter Delay Compensation (TDC) mechanism introduced in CF, see Section 3.3. For data-phase to arbitration-phase bit rate ratios below approximately 9, CF accepts the same oscillator tolerance as CC [6]. Below this threshold, the arbitration-phase resynchronization condition remains the binding constraint, so the higher data-phase bit rate does not impose a tighter oscillator requirement.

CF also strengthens the error detection architecture relative to CC. CC's CRC-15 polynomial preserves a Hamming distance of 6 only up to a certain frame length, making it insufficient for CF's longer payloads. CF replaces it with a 17-bit polynomial for frames up to 16 data bytes and a 21-bit polynomial for frames up to 64 bytes, preserving the Hamming distance of 6 across the extended payload range and guaranteeing detection of any five or fewer bit errors [2].

CF also addresses a gap in CC's stuff-bit handling: in CC, dynamic stuff bits are excluded from the CRC calculation, allowing rare two-bit errors that create or destroy a stuff condition to go undetected. CF closes this by including dynamic stuff bits in the CRC data feed and introducing the Stuff Bit Count (SBC) field as

an independent check on the number of inserted stuff bits. The combined effect of these changes extends the CRC field from 16 bits in CC to 28 bits (CRC17) or 33 bits (CRC21). With 12 or 17 more bits required to accidentally match the expected CRC field, the residual error probability for this class of faults is reduced by several orders of magnitude [7].

1.1.3 Everllence's Current CAN Controller

The starting point for this project is Everllence's internally developed current CC controller (`can_bus_controller`). The controller is implemented in VHDL and has been integrated into a production IO-extender FPGA design. It supports CC frames with both 11-bit (base) and 29-bit (extended) identifiers at bit rates up to 500 kbit/s, and has been verified through hardware bring-up on physical CAN buses. The initial version was developed by the author of the present document during an internship at Everllence and has since been extensively modified by other engineers at the company.

The top-level wrapper for `can_bus_controller` instantiates a combined TX/RX frame Finite State Machine (FSM), a bit timing generator, a dynamic bit stuffer, a CRC-15 engine, and two Avalon-ST converters for frame serialization and deserialization.

The central component is the orchestrating FSM that handles both transmission and reception in a single process. It manages frame arbitration, bit-level TX and RX, stuff-bit error checking, CRC validation, acknowledgment (ACK) handling, and error flag generation. Fault confinement (error active, error passive, bus off node state transition) is handled in a separate process within the main FSM entity.

While the `can_bus_controller` is functional for CC, several areas of its design would require significant rework to support CF. The main limitations are listed below.

- **Single bit rate domain**: The controller operates at a single bit rate with no data-phase switching or TDC support (Section 3.3).
- **Dynamic bit stuffing only**: The stuffer has no mechanism for fixed bit stuffing or SBC generation (Section 3.4).
- **Single CRC engine**: The controller has a single CRC engine, where CF compliance requires three running in parallel with separate data feed paths (Section 3.5).
- **Coupled fault confinement**: Fault confinement is coupled into the frame FSM with no clean sublayer boundary (Section 3.6). ISO 11898-1 defines it as a distinct cross-cutting entity, and separating it improves both conformance to the standard's reference model (Section 3.1) and testability.
- **Format-dependent frame structure**: CC and CF frames diverge in the control and data phases, where CF introduces format-specific fields with no Classic equivalent (Section 3.2). The existing FSM has no clean mechanism to handle this divergence across four frame variants.

The changes required across bit timing, bit stuffing, CRC, and frame format handling are pervasive enough to justify a clean-slate redesign.

1.2 Available Third-Party CAN FD IP Cores

Before committing to an in-house redesign, the available CF controller IP cores were evaluated. Table 2 summarizes the candidates, spanning both open-source and commercial offerings.

1.2.1 Open-Source Implementations

CTU CAN FD [8] is the only mature open-source CF controller available as synthesizable HDL. Developed at the Czech Technical University in Prague, it is written in VHDL, licensed under MIT, and has been conformance-tested against ISO 16845-1 [9]. The controller includes a full TX and RX pipeline with up to four TX buffers, acceptance filtering, timestamping, and a register interface with DMA support.

1.2.2 Commercial Implementations

Bosch M_CAN [10] is the reference CF controller developed by Bosh. M_CAN is the IP core embedded in most automotive micro controllers. It is licensed under a non-disclosure agreement with per-design royalty fees.

AMD/Xilinx CAN FD [11] is a soft IP core included in the Vivado Design Suite. It provides an AXI4-Lite register interface with up to 32 acceptance filters, TX mailboxes, and RX FIFOs. It is device-locked to AMD/Xilinx FPGAs and cannot be ported to other targets.

Technology-independent RTL cores. CAST CAN FD [12] is a technology-independent CF IP core licensed per-design with an upfront fee. Major vendors including Synopsys (DesignWare) and Cadence offer similar Application-Specific Integrated Circuit (ASIC)-targeted cores under commercial licensing programs.

Table 2: Survey of available CF controller IP cores.

Implementation	Source	License	Scope	Conformance Tested
CTU CAN FD [8]	VHDL (MIT)	MIT	Full node	Yes
Bosch M_CAN [10]	Closed	Per-design royalty	Full node	Yes (reference)
AMD CAN FD [11]	Closed	Vivado-included	Full node	Yes
CAST CAN FD [12]	Closed	Per-design fee	Full node	Yes

1.2.3 Rationale for In-House Development

None of these solutions satisfies Everllence's combined requirements. The disqualifying factors span IP ownership, verification authority, architectural scope, integration with existing infrastructure, and platform independence - each addressed in turn below.

- **IP ownership and supply chain independence:** Everllence's engine controllers carry service commitments of up to thirty years. Commercial IP cores introduce a licensing dependency on an external vendor over that full horizon - vendors may discontinue support, change licensing terms, or be acquired. Owning the RTL outright eliminates this exposure and ensures that the design can be maintained, ported, and modified without third-party approval for the full product lifetime. The open-source CTU CAN FD avoids the licensing risk, but using it still means adopting a codebase whose architecture, naming conventions, and design decisions were made for a different context.
- **Verification authority:** In safety-critical domains, the verification evidence must be traceable from standard requirements to RTL assertions and testbench results. Adopting a third-party core means inheriting its verification artifacts rather than producing them. Everllence's verification methodology requires full control over the verification plan, the testbench architecture, and the assertion coverage.
- **Architectural scope:** All available IP cores implement a complete CAN node - TX and RX pipelines, message memory, acceptance filtering, buffer management, register interfaces, and in

some cases DMA controllers. Everllence's application requires only the protocol engine - the module that converts between byte-level frame data and the serial bus - which is also the scope of `can_bus_controller`. The higher-level buffering and filtering logic already exists in Everllence's FPGA infrastructure. Adopting a full-node IP core would introduce unnecessary complexity and area overhead, and stripping the unused subsystems to fit the existing architecture offsets the benefit of using a pre-built core.

- **Integration with existing infrastructure:** Everllence's FPGA designs use a specific Avalon-ST streaming interface for inter-module communication and established conventions for signal naming and module boundaries. A third-party core would require an adaptation layer to bridge its native interface to the existing infrastructure. An in-house design would use Everllence's interface conventions natively, eliminating this integration overhead.
- **Platform independence:** The AMD/Xilinx CAN FD core is locked to Xilinx devices. The Bosch M_CAN and other commercial cores are delivered as technology-specific netlists or encrypted HDL for a particular target. The in-house design is written in portable VHDL-93, synthesizable on any FPGA platform ensuring that the IP remains usable if Everllence changes FPGA vendors.

1.3 Problem Statement

The architectural limitations of the existing controller (Section 1.1.3) and the unsuitability of available third-party IP cores (Section 1.2.3) together motivate a clean-slate CF protocol controller conforming to ISO 11898-1. No existing solution combines full IP ownership, a targeted data-link-layer scope matching Everllence's integration requirements, and native compatibility with Everllence's Avalon-ST interface conventions and VHDL Code Standard. The design described in this report addresses that gap directly.

1.4 Objectives

1. Implement a CC/CF protocol controller in VHDL, compliant with ISO 11898-1 [3].
2. Establish a structured requirements framework with traceability from ISO 11898-1 to testbench evidence, covering all implemented modules.
3. Produce an RTL design integrated via Avalon-ST interfaces into Everllence's existing FPGA infrastructure.

2 Requirements

This section establishes the constraints that bound the design before any architectural decisions are made. Everllence's coding standard, tooling, and existing FPGA infrastructure set the implementation framework, while the protocol requirements derived from ISO 11898-1 define the functional obligations of the controller.

2.1 Code Standard, Tooling, and Design Constraints

2.1.1 Tools and Language

The RTL source is implemented in VHDL-93. Everllence's synthesis toolchain uses Quartus Prime [13], which does not fully support VHDL-2008 constructs in synthesis, making VHDL-93 the practical upper bound for synthesizable RTL. Testbenches are written in VHDL-2008 for the OSVVM verification framework [14]. SystemVerilog with UVM (Universal Verification Methodology) is the dominant industry alternative for RTL implementation and verification at this scale. The choice here follows company convention

rather than a project-level technical comparison. Riviera-PRO [15] is used for simulation. Sigasi [16] is used for linting and language-aware editing. Waveform figures are captured in GTKWave [17], timing diagrams are drawn in WaveDrom [18], and architecture diagrams in Mermaid [19].

Claude Code [20] was used for prose editing and VHDL review. Technical content, design decisions, and all source files are the author's own work.

2.1.2 Integration Requirements

1. **Avalon-ST user interface:** The CAN controller's external interface to the host system must use the Avalon-ST streaming protocol [21] (data, valid, ready, sop, eop). The requirement applies specifically to the boundary between the CAN controller and its user.
2. **IP library CRC block:** Everllence maintains a reusable, parameterized CRC generator (`gen_crc`) in its IP library. This module must be used for all CRC computations.

2.1.3 VHDL Code Standard

1. **Entity port types:** Entity ports are restricted to `std_logic`, `std_logic_vector`, and records or arrays of these types. Modes are restricted to `in` and `out`.
2. **Naming conventions:** A mandatory prefix/suffix scheme applies to all VHDL identifiers: types (`t_`), constants (`c_`), generics (`gc_`), processes (`p_`), functions (`f_`), packages (`pk_`), state variables (`s_`), entity inputs (`_i`), and entity outputs (`_o`).
3. **RTL design rules:** Synchronous processes must be sensitive to the clock only. Reset must be synchronous and initialize all control registers. FSMs are preferably implemented as single-process designs where all signal assignments are derived from the current state.
4. **Testbench requirements:** Testbenches must follow a black-box testing model and test cases must be ordered as: reset tests first, then a normal-usage test, then all remaining tests.

2.2 From Specification to Structured Requirements

The requirements engineering process addressed two key objectives [22]:

1. Extracting a clear and actionable set of requirements that could serve as a starting point for the design phase.
2. Establishing a clear, traceable link between the ISO 11898-1 specification and the verification environment.

Both objectives are complicated by the source material: normative requirements are distributed across subsections, bundled into compound clauses, and repeated from transmitter and receiver perspectives, interspersed with informative rationale prose.

The AI-augmented pipeline shown in Figure 2 was designed to address these extraction challenges systematically. The first step was converting the ISO 11898-1 PDF to Markdown - a format that can be efficiently searched and ingested by Large Language Models (LLMs). The resulting Markdown file was then fed to a Claude Sonnet 4.6 LLM agent, which was prompted to extract all normative statements - sentences containing words like "shall", "should", "must", and their corresponding negations.

This process yielded a raw set of 168 normative statements linked to the ISO standard sections from which they were extracted. The normative set was then manually reviewed, consolidated, and distilled into a final set of 37 requirements (reproduced in Section B).

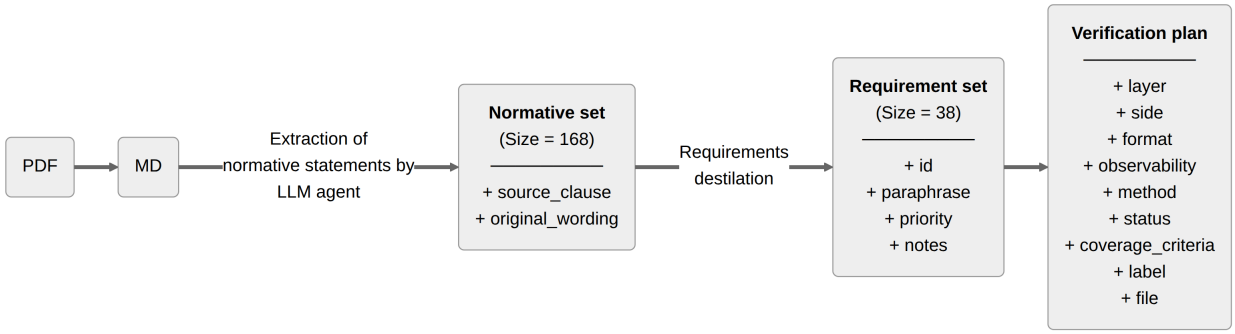

Figure 2: Pipeline generating the `verification_plan.toml` artifact from the ISO 11898-1 standard.

The requirements set is stored as a TOML file, one entry per requirement. Direct LLM editing of large structured files is unreliable - prone to silent entry deletion, field hallucination, and syntax corruption. To make iterative AI-assisted refinement of the plan viable, a custom Model Context Protocol (MCP) server was developed alongside it. The server exposes query, insert, update, and delete operations, together with bulk update and statistics utilities, as structured tool calls. Each write operation targets a single requirement entry and validates field values against the schema before committing. This bounds any model error to one requirement and prevents malformed data from reaching the file. Each requirement entry contains the following fields:

- **`source_clause`**: Links every requirement back to the ISO 11898-1 clause from which it was distilled, enabling the requirements set to be audited against the standard.
- **`original_wording`**: Verbatim ISO text for the relevant clauses. Preserving the source wording prevents paraphrase drift and provides a fallback for resolving ambiguity during implementation.
- **`paraphrase`**: A concise, implementer-facing restatement of the requirement. Where the ISO prose bundles multiple obligations into a single clause, the paraphrase enumerates them as numbered sub-claims, each independently verifiable.
- **`priority`**: A three-level rating - P1 (need-to-have, derived from "shall" obligations and core correctness), P2 (judged non-critical for this application: the bus functions in Everllence's target use case without these, though they are part of full standard compliance), or P3 (optional, derived from "should" clauses or implementation-dependent features). The priority drove the implementation sequence and scope decisions.
- **`notes`**: Any residual clarifications not captured by the paraphrase - implementation constraints, out-of-scope markers, or known ambiguities. May be empty.

Of the 37 requirements, 30 are rated P1, four are P2, and three are P3. Each non-P1 rating reflects a deliberate scoping decision - the rationale for each is given in Table 3.

Table 3: Priority demotion rationale for all requirements not rated P1.

ID	Topic	Priority	Demotion rationale
REQ-002	LLC TX request and abort timing	P2	The 2-SOF processing window is a responsiveness guarantee, not a correctness constraint. A node that transmits eventually but outside this window sends valid frames.
REQ-011	Remote frame	P2	A data-only node is a valid CC/CF implementation. Remote frame support is a distinct feature subset not required for basic interoperability.

ID	Topic	Priority	Demotion rationale
REQ-015	ESI bit transmission	P2	ESI communicates the node's error state as an informational signal. Incorrect ESI does not abort a frame or trigger a protocol error at any receiver.
REQ-035	Error signaling enable	P2	Error signaling itself is covered by P1 requirements. This requirement concerns only the existence of a configurable disable mode, which is an optional operational feature.
REQ-004	Frame acceptance filtering	P3	Acceptance filtering is absent from Everllence's current controller, so no regression concerns. The only protocol-relevant filter - suppressing loopback of transmitted frames - is covered by the design.
REQ-034	Shared memory consistency	P3	The "may" language makes shared memory use optional.
REQ-036	DLC padding	P3	Padding with 0xCC applies only when the implementation exposes a configurable maximum-data-byte restriction. The feature may be waived entirely if that restriction is not implemented.

The distillation step that followed - consolidating 168 raw normative statements into 37 prioritized, independently verifiable requirements - required substantial manual effort that the AI could not replace. Deciding which statements address the same underlying obligation, how to bound each requirement so that it is independently verifiable, and how to assign priority in a way that is defensible against the standard text are judgment calls that depend on understanding the protocol at an implementation level. REQ-026 illustrates the challenge well - the corresponding extracted normative statements are:

1. *"Only one synchronization within one bit time (between two sample points) shall be allowed. After an edge is detected, synchronizations shall be disabled until the next time the bus state detected at the sample point is recessive."*
2. *"An edge shall cause synchronization only if the bus state detected at the previous sample point was recessive. If a transmitter uses transmitter delay compensation, also the first detected edge from recessive to dominant after the sample point of the CRC delimiter shall cause synchronization. An edge with a positive phase error shall not cause synchronization in a node sending a dominant bit."*
3. *"Hard synchronization shall be performed: on edges during inter-frame space (with the exception of the first bit of intermission); when a node is in bus-integration state; at the edge between the FDF bit and the following dominant res bit inside an FD frame; when a node is a receiver of an XL frame, at the edge between the XLF bit and the following dominant resXL bit; at the edge between the DH1 and DH2 bits and the DL1 bit; at the edge between the DAH or AH1 bit and the AL1 bit."*
4. *"All other recessive-to-dominant edges fulfilling rules a) and b) shall be used for resynchronization with one exception: a node transmitting an FD frame or an XL frame shall not synchronize while it transmits the data phase of that frame."*
5. *"After a hard synchronization, the bit time shall be restarted with Sync_Seg completed. Hard synchronization shall not be limited by the synchronization jump width."*
6. *"When the magnitude of the phase error is less than or equal to the synchronization jump width, the effect of a resynchronization shall be the same as a hard synchronization."*
7. *"When the magnitude of the phase error is larger than the synchronization jump width: if positive, Phase_Seg1 shall be lengthened by the synchronization jump width; if negative, Phase_Seg2 shall be shortened by the synchronization jump width."*

Statements 3 and 4 each embed CAN XL elements within otherwise in-scope obligations, which must be excluded without dropping the surrounding CB, CE, FB, and FE rules. The remaining statements interleave hard sync and resync rules across four sub-clauses, requiring the two mechanisms to be separated and their interaction made explicit. The resulting paraphrase is presented below, with the remaining requirement fields in Table 4.

Paraphrase:

1. Phase error: the time interval between an R-to-D edge and the Sync_Seg boundary of the current bit time. Positive when the edge falls after Sync_Seg (late), negative when before (early).
2. Qualifying edge: R-to-D, previous SP is R.
3. SJW is the maximum per-resynchronization adjustment to Phase_Seg1 or Phase_Seg2. Qualifying edges cause resynchronization. If |phase error| ≤ SJW: same effect as hard synchronization. If phase error > SJW: Phase_Seg1 += SJW. If phase error < -SJW: Phase_Seg2 -= SJW.
4. One synchronization per bit time, re-enabled after the next R SP.
5. Hard synchronization: IFS edges (except first intermission bit), bus-integration state, and FDF-to-res transition in FD frames. Restarts bit time with Sync_Seg completed.

Table 4: REQ-026 distilled from seven extracted normative statements.

Field	Value
ID	REQ-026
Source	§7.3.5.1, §7.3.5.2, §7.3.5.3, §7.3.5.4, Figure 33
Priority	P1
Notes	RTL is stricter than ISO: sync is suppressed unconditionally when transmitting. Safe since the transmitter is the timing source.

3 CAN Classic and CAN FD Protocol Overview

The 37 requirements distilled in Section 2 define what must be implemented and verified - but they also function as a structured map to the protocol, since every requirement points to a mechanism that must be understood before implementation can begin. Those mechanisms - the sub-layer model, frame formats, bit timing, bit stuffing, CRC, and error handling - are covered here, each cross-referenced to the relevant REQ-NNN entries. Readers familiar with ISO 11898-1 may skip to Section 4.

3.1 Layered Reference Model

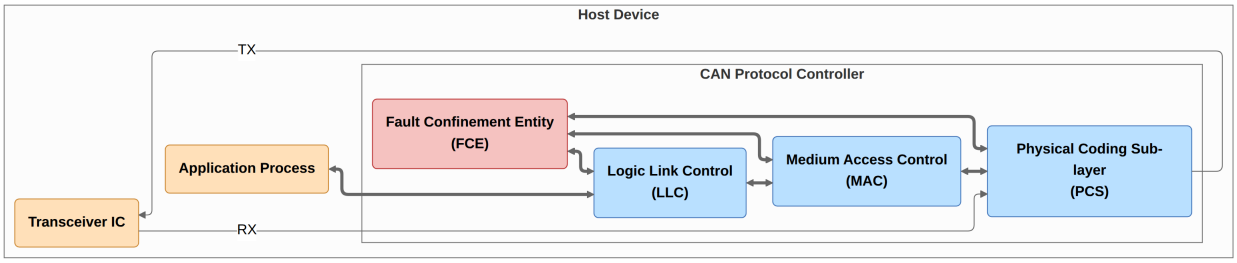

Figure 3: ISO 11898-1 CAN node reference model showing the LLC, MAC, PCS sub-layers and cross-cutting FCE.

ISO 11898-1 structures the CAN node reference model into three functional sub-layers - LLC and MAC in the data link layer, PCS in the physical layer - and a cross-cutting Fault Confinement Entity (FCE) [3, Fig. 4] (see Figure 3):

- **LLC (Logical Link Control)**: Acceptance filtering (selecting received frames by identifier), overload notification (delaying the next frame when internal conditions require it), and recovery management (retransmission on error or lost arbitration), and supplying frames to the MAC.
- **MAC (Medium Access Control)**: Encodes and decodes the frame bit-by-bit, performing bit stuffing and destuffing, CRC generation and checking, error detection and signaling, acknowledgment handling, and medium access arbitration.
- **PCS (Physical Coding Sublayer)**: Bit timing and bus sampling (segmenting each bit time and reading the bus at the sample point), clock synchronization, and the TX/RX interface to the physical transceiver.
- **FCE (Fault Confinement Entity)**: Escalating the node's error state from error active through error passive to bus off as transmit and receive error counts accumulate.

3.2 Frame Types and Formats

CAN frames may carry either an 11-bit base identifier or a 29-bit extended identifier, giving four frame formats: CBFF (CAN Classic Base Frame Format), CEFF (CAN Classic Extended Frame Format), FBFF (CAN FD Base Frame Format), and FEFF (CAN FD Extended Frame Format) (REQ-037). The four frame formats are depicted in Figure 4 along with the Active/Passive Error Flags (AEF/PEF). CBFF and CEFF additionally support Remote Frame (RF) variants, giving six frame format types in total.

All frame formats opens with a D SOF bit that triggers hard synchronization (Section 3.3) in all receiving nodes, followed by the arbitration, control, data, and CRC fields, an ACK slot, and a seven-bit EOF delimiter. The RTR bit is D for data frames and R for RF. The IDE bit distinguishes base frames from extended frames. Fixed-polarity form bits (SRR, r0, r1) carry no protocol instruction. The DLC encodes the number of data bytes in the payload (REQ-031). The ACK slot carries a dominant bit driven by every receiver that has validated the CRC. Each frame is followed by the interframe space - intermission (INT), suspend transmission (ST, error passive transmitters only), and bus idle (REQ-008).

CFFF share the same arbitration phase structure, with the FDF bit signaling an CF format when R. The CF control field contains a reserved form bit (res) alongside BRS and ESI. The DLC retains its 4-bit width but uses a non-linear mapping above 8 bytes, extending the maximum payload to 64 bytes (REQ-031). The BRS (Bit Rate Switch) bit controls the transition to the data-phase bit rate. When BRS is R the bus switches to the faster data rate immediately after the BRS sample point and returns to the nominal rate at the CRC delimiter (REQ-030). The ESI (Error State Indicator) bit reflects the transmitting node's fault-confinement state: a node in error passive state shall transmit ESI recessive (REQ-015). The CFFF CRC field is additionally prefixed by a Stuff Bit Count (SBC) - a Gray-coded count of dynamic stuff bits with a parity bit (REQ-016).

3.3 Bit Timing

Every CAN bit period is divided into four non-overlapping time segments measured in Time Quanta (TQ), where one TQ equals the system clock period multiplied by a programmable integer prescaler, see Figure 5. CF extends CC with an independently configured data-phase bit rate. A CF node therefore maintains two independent sets of segment parameters, one for the nominal rate and one for the data rate (REQ-024). The segments are listed described below.

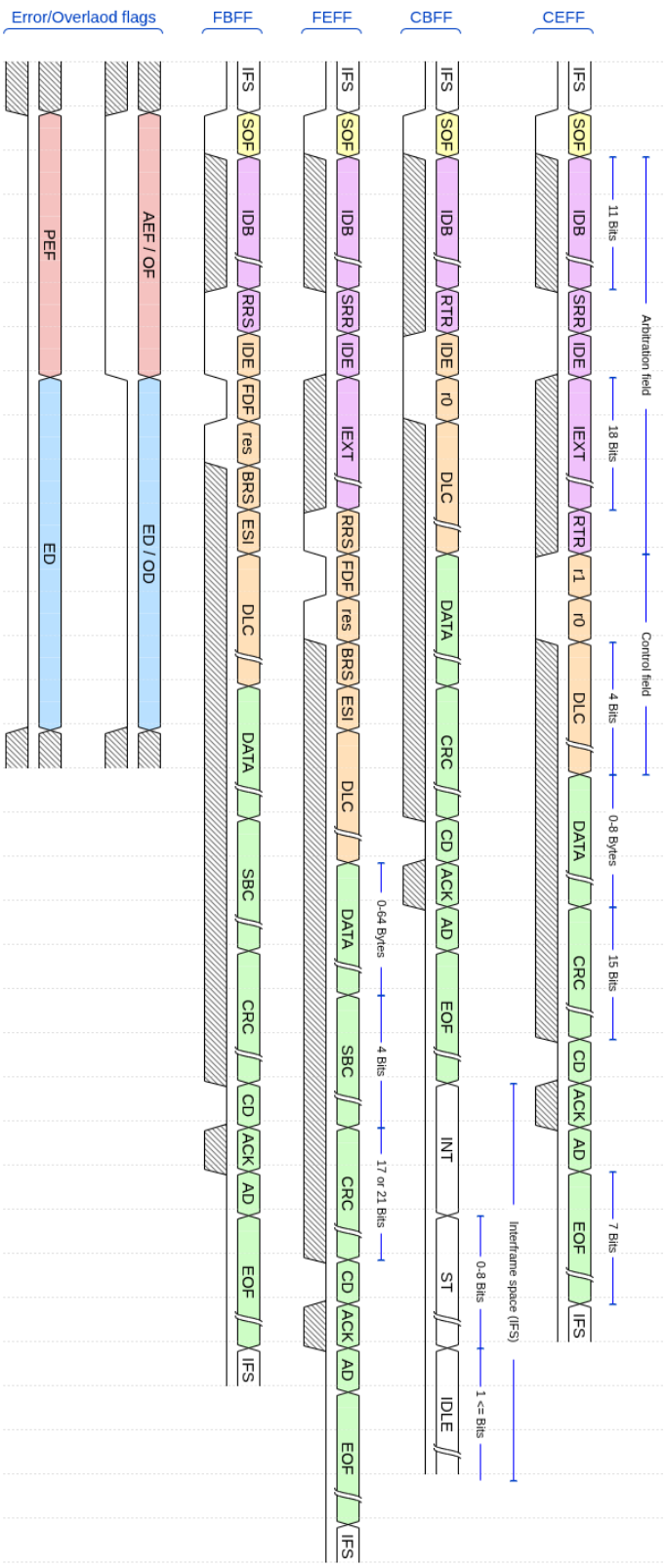

Figure 4: CAN frame formats (CBFF, CEFF, FBFF, FEFF) and the error and overload flags, with field widths annotated per ISO 11898-1. The bus waveform below each format indicates the level of fixed-polarity protocol bits. Hatched regions indicate variable-content fields.

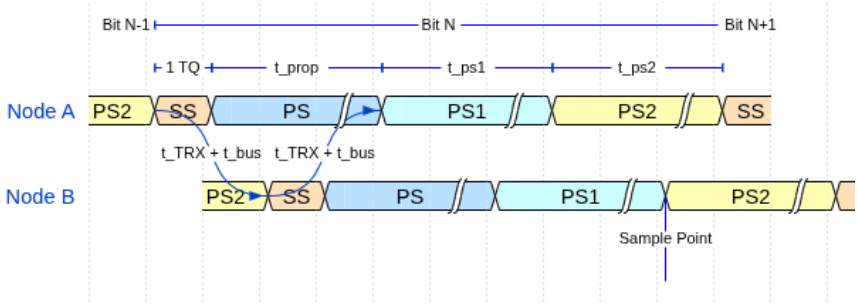

Figure 5: CAN bit time segments (SS, PS, PS1, PS2) and Sample Point (SP). The two-node layout illustrates the round-trip propagation delay (t_TRX + t_bus each way) that PS must cover for correct arbitration.

- **Sync Segment (SS)**: one TQ. The segment at which a recessive-to-dominant edge is expected when the node is synchronized.
- **Propagation Segment (PS)**: guarantees that a bit driven by any node reaches all others before the sample point, enabling correct arbitration. PS shall be programmed to at least twice the one-way propagation delay: $2 \times$ (t_TRX + t_bus), see Figure 5.
- **Phase Segment 1 (PS1)**: immediately precedes the sample point. Can be lengthened by the resynchronization mechanism to absorb positive phase errors.
- **Phase Segment 2 (PS2)**: follows the sample point to the end of the bit. Can be shortened to absorb negative phase errors.

The bus is sampled at the sample point (SP) which falls at the PS1 / PS2 boundary.

3.3.1 Synchronization

In a synchronized receiving node, edges arrive within SS. An edge outside SS carries Phase Error (PE) and triggers synchronization to realign the sample point at the PS1/PS2 boundary. During the frame, resynchronization on each qualifying recessive-to-dominant edge adjusts the bit time based on the PE relative to SS: a positive PE lengthens PS1 by up to the Synchronization Jump Width (SJW), and a negative PE shortens PS2 by up to SJW. Hard synchronization, triggered by the SOF dominant edge and the FDF-to-res dominant edge, restarts the bit time with SS completed. Only one synchronization is permitted per bit time (REQ-026). Figure 6 illustrates how two consecutive mid-frame synchronization events align an unsynchronized receiving node to the transmitter on the bus.

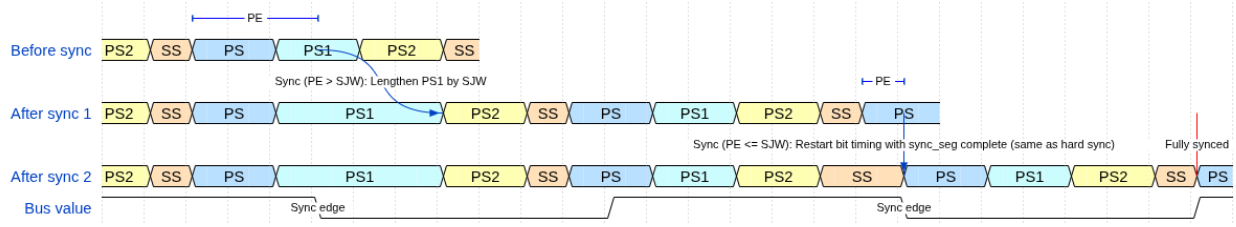

Figure 6: Resynchronization over two successive sync edges. PE is the phase error relative to SS. SJW is the Synchronization Jump Width.

3.3.2 Transmitter Delay Compensation

When the CF data phase begins, the transceiver loopback delay may span multiple data-phase bit times, necessitating a delay before sampling to compensate for in-flight bits (REQ-025). TDC measures this delay in the nominal-rate control field: from when the res bit goes dominant on TX to when the edge arrives on

RX. The Secondary Sample Point (SSP) is then placed at the measured delay plus a programmable offset, firing before the SP in each bit time so that bit errors can be reacted upon at the SP. For the initial data-phase bits where the loopback has not yet returned, the SSP is suppressed. From the first valid SSP onward, SSP monitoring replaces SP monitoring for the remainder of the data phase, see Figure 7.

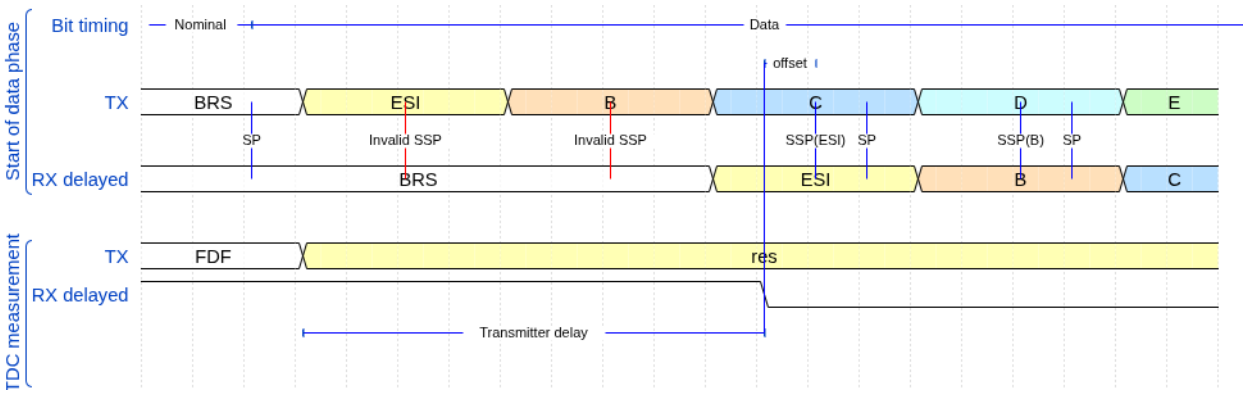

Figure 7: Transmitter Delay Compensation. Top: the first data-phase bits have no valid SSP while the loopback is still in transit. Once the loopback returns, the SSP is placed at the measured transceiver delay plus a programmable offset, firing before the SP in each bit time. Bottom: the transceiver delay is measured from when the res bit goes dominant on TX to when the edge arrives on RX.

3.4 Bit Stuffing

Bit stuffing ensures sufficient transitions on the bus for receiver clock synchronization. CC applies dynamic stuffing from SOF through the CRC field: after five consecutive bits of the same polarity, the transmitter inserts one complement Stuff Bit (SB) and the receiver discards it (REQ-018). CAN FD retains dynamic stuffing through the arbitration and data fields, then switches to fixed stuffing in the CRC field. Fixed Stuff Bits are inserted at fixed positions - before the SBC field and after every fourth CRC bit - each the inverse of the preceding bit, guaranteeing transitions regardless of the CRC data pattern. A separate Stuff Bit Count (SBC) field, Gray-coded with a parity bit, enables receivers to verify the number of dynamic stuff bits inserted in the frame (REQ-016, Figure 8).

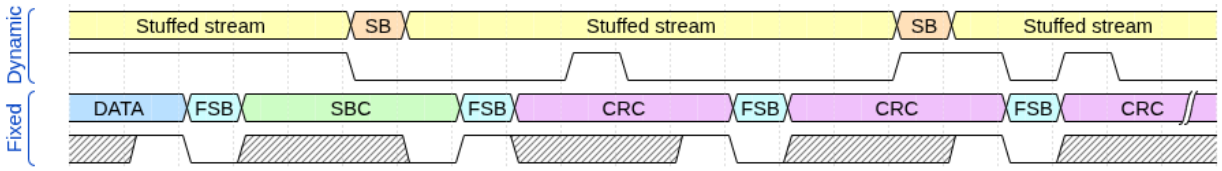

Figure 8: Dynamic and fixed bit-stuffing examples showing stuff bit placement for both encoding modes. The waveform below each row shows the resulting bus signal.

3.5 Cyclic Redundancy Check

The transmitter divides the frame bit stream by a fixed generator polynomial and appends the remainder as the CRC field. Receivers repeat this operation and flag a mismatch as a CRC error. The polynomial and field length depend on frame type and data payload length (REQ-006) as listed below.

- **CRC-15**: Used for all CC frames. The CRC accumulates over SOF, arbitration, control, and data fields with SBs excluded (REQ-013).
- **CRC-17**: Used for FD frames with data payloads up to 16 bytes (DLC 0-10).
- **CRC-21**: Used for FD frames with data payloads from 20 to 64 bytes (DLC 11-15).

The key asymmetry between CC and FD concerns the CRC data feed. For FD frames, dynamic stuff bits up to and including the data field are included in the CRC computation, along with the SBC field itself. Fixed stuff bits are excluded from the CRC computation in both CC and FD frames (REQ-013). The CRC field is terminated by a recessive CRC delimiter bit (REQ-017).

3.6 Error Detection and Fault Confinement

Every CAN node monitors the bus for five categories of error (REQ-021):

- **Bit error**: A transmitter reads back a polarity different from what it drove. Exceptions: a recessive non-stuff bit overridden during arbitration and a recessive bit in the ACK slot.
- **Stuff error**: Six consecutive bits of the same polarity in frame fields where dynamic bit stuffing applies.
- **CRC error**: Received checksum does not match the locally recomputed value.
- **Form error**: A fixed-format field contains an illegal bit value.
- **Acknowledgment error**: A transmitter receives no dominant ACK bit from any receiver (REQ-014).

Detection of any error causes the detecting node to abort frame transmission or reception and start transmitting an error flag (REQ-022). An error active node transmits an Active Error Flag (AEF) of six consecutive dominant bits. An error passive node transmits a Passive Error Flag (PEF) of six consecutive recessive bits instead (REQ-007). Both are followed by an eight-bit recessive error delimiter. Transmission of an AEF flag will trigger bit-stuffing errors in the receiving nodes on the bus, causing these nodes to transmit their own error flags.

The FCE tracks each node's error history through the Transmit Error Counter (TEC) and Receive Error Counter (REC). Counter increments and decrements follow the rules defined in REQ-028. A node begins in error active, transitions to error passive when either counter exceeds 127, and to bus off when TEC exceeds 255 (REQ-029). An error-passive transmitter is exempt from the TEC increment on an ACK error, preventing an isolated node from escalating to bus-off through failed acknowledgments alone. In the bus off state, the node ceases all bus activity and shall not influence the bus (REQ-027) until 128 sequences of 11 consecutive recessive bits are observed, after which TEC and REC are reset and the node returns to error active. A host-initiated reset also returns the FCE to its initial state immediately (REQ-029).

4 Verification Plan

The verification plan adds five classification dimensions to each of the 37 requirements, driving both the module architecture and the testbench design. The plan was populated using MCP introduced in Section 2. The five classification dimensions fall into two groups. Design-facing dimensions inform the module architecture. Verification-facing dimensions inform the verification strategy and testbench design. Each field is summarized in the bullets below and described in detail in the following sections.

- **Design-facing**: `layer`, `side`, and `format_applicability` - determining module ownership, TX/RX path decomposition, and which frame formats each requirement applies to.
- **Verification-facing**: `observability` and `verification_method` - determining whether internal state is required and specifying the verification technique.

4.1 Layer

The `layer` field assigns each requirement to the sub-layer that owns it: LLC, MAC, PCS, or FCE. A fifth label, `system`, marks requirements that are inherently multi-layer or multi-node and need either an integrated testbench or a two-node simulation. REQ-020 (arbitration loss) illustrates the latter: verifying it requires MAC, PCS, and FCE cooperating in a full node, plus a second node driving dominant while the Device Under Test (DUT) drives recessive.

4.2 Side

The `side` field classifies each requirement as transmitter, receiver, or both, reflecting the ISO standard's practice of specifying TX and RX obligations separately. It scopes the testbench to the corresponding drive mode. REQ-025 (TDC) illustrates a TX-only requirement - measuring transceiver loop delay and positioning the SSP only apply to transmitting nodes.

4.3 Format Applicability

The `format_applicability` field records which of the in-scope frame formats (CBFF, CEFF, FBFF, FEFF) each requirement applies to. Because the formats differ in stuffing mode, CRC polynomial, and control field structure, a requirement that applies only to CF frames implies stimulus with FDF=1 and DLC values covering both sides of the CRC-17/CRC-21 boundary, while one that applies to all four formats requires a configuration for each.

4.4 Observability

The `observability` field classifies each requirement as either black-box or white-box, relative to the module boundary of the owning layer:

- **Black-box**: Can be verified purely through the module's observable port signals.
- **White-box**: Verification requires direct observation of the module's internal state.

This classification drives the testbench architecture. Black-box requirements can be verified by driving the module inputs and checking the output signals. White-box requirements require a parallel reference model or direct observation of internal signals. REQ-006 (CRC polynomial selection) is representative: the polynomial in use - CRC_15, CRC_17, or CRC_21 - is configured inside `can_mac_crc` and never exposed at the module boundary. Verification uses a reference model that independently computes the expected CRC for each format and DLC combination and compares it against the transmitted sequence.

4.5 Verification Method

The `verification_method` field specifies how each requirement will be checked. Four methods are used:

- **`simulation`**: Assertion procedures in a testbench.
- **`code_inspection`**: RTL source review.
- **`waveform_inspection`**: Manual review of simulation output.
- **`coverage`**: A functional coverage bin confirming a specific condition was exercised.

Combinations are valid when multiple sub-claims within one requirement each call for a different method. REQ-018 (bit stuffing) illustrates this: `simulation` assertions verify that stuff bits are inserted and removed at the correct positions, while `coverage` bins confirm that the edge case of five consecutive identical bits landing at a field boundary was exercised at least once.

4.6 Traceability: Label and File

The `label` and `file` fields establish a direct, navigable link from each requirement to its verification artifact. `file` identifies the testbench or RTL source file responsible for covering the requirement. `label` identifies a specific named procedure, assertion, or coverage ID within that file.

4.7 Status

The `status` field (`not_started`, `in_progress`, `complete`) records requirement closure state explicitly, allowing partial progress to be tracked.

4.8 Verification Plan Summary

Table 5 shows the 37 requirements distributed across layer, side, format scope, and observability. MAC carries 20 of the 37, with 18 white-box, reflecting the breadth of frame-encoding logic that requires bit-level state access to verify. FCE is the opposite: both requirements are black-box, since fault-confinement state transitions are fully observable through the node's error-state output signals. The complete verification plan is reproduced in Section B as two separate tables linked by common IDs.

Table 5: Requirement distribution by layer, side, format scope, and observability. n = total. FS = format-specific (not applicable to all four frame formats). BB = black-box. WB = white-box.

Layer	Requirements	n	TX	RX	Both	FS	BB	WB
MAC	REQ-006, REQ-008, REQ-010–013, REQ-015–019, REQ-021–023, REQ-030–032, REQ-034–035, REQ-037	20	3	0	17	5	2	18
LLC	REQ-001–005, REQ-036	6	3	1	2	0	3	3
PCS	REQ-024–027	4	1	0	3	1	1	3
FCE	REQ-028–029	2	0	0	2	0	2	0
System	REQ-007, REQ-009, REQ-014, REQ-020, REQ-033	5	1	0	4	0	2	3
Total		**37**	**8**	**1**	**28**	**6**	**10**	**27**

5 Design and Architecture

This section presents the key architectural design decisions driving the implementation. It covers the architecture module decomposition, the LLC frame format, and the structural choices that govern the MAC sub-layer FSM.

The design maps each ISO 11898-1 refference model sub-layer to a dedicated module, as shown in Figure 9. This allows the module boundaries to align directly with in verification targets, each requirement points unambiguously to the responsible implementation unit, and each module can be exercised in isolation. The primary data path runs from `can_llc` through `can_mac` to `can_pcs`, with `can_fce` sitting outside the path as a cross-cutting entity. `can_mac` bundles the four MAC sub-modules into a single sub-layer wrapper. `can_mac_pcs_fce` further combines `can_mac`, `can_pcs`, and `can_fce` into a synthesizable integration

target. A centralized types package (`pk_can_types`) defines all protocol constants, interface records, and reset values shared across modules. The responsibilities of the individual modules are listed below.

- **`can_llc`:** Implements the LLC sub-layer. It has a host-facing and a `can_mac`-facing Avalon-ST interface. Applies acceptance filtering on frames received from `can_mac` and handles retransmission when `can_mac` reports arbitration loss or error/overload. It converts between the host and MAC-facing LLC frame formats described in Section 5.2.
- **`can_mac_fsm`:** Orchestrates frame TX and RX across all four in-scope formats, controls the back-pressure ready signal on the `can_mac_ser` interface, drives `can_mac_bs` and `can_mac_crc`, and commits the next bit to the `can_pcs`. Streams received frames to `can_llc` over Avalon-ST.
- **`can_mac_ser`:** Receives the LLC frame over Avalon-ST from `can_llc` and presents a serialized bit stream to `can_mac_fsm` (using a valid/ready handshake) along with frame metadata bits (DLC, ESI, FDF, BRS, FTYP).
- **`can_mac_bs`:** Performs dynamic and fixed bit stuffing/de-stuffing and generates the SBC field for CF frames.
- **`can_mac_crc`:** Runs CRC-15, CRC-17, and CRC-21 engines in parallel, selecting the output based on frame format and DLC.
- **`can_pcs`:** Applies bit timing, generates SP and SSP strobes, performs hard synchronization and resynchronization, and manages the TX/RX interface to the transceiver.
- **`can_fce`:** Tracks transmit and receive error counts, manages error-active/error-passive/bus-off state transitions, and signals bus-off entry and recovery.

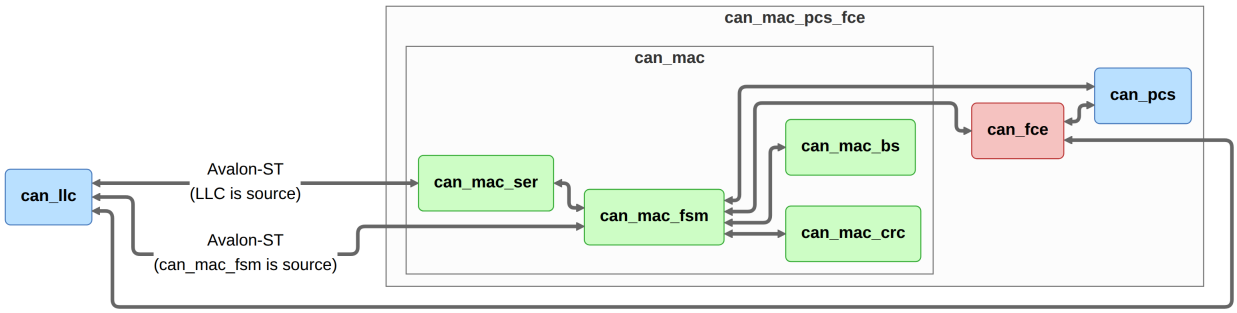

Figure 9: Architecture module decomposition. Wrapper modules are shown as bounding boxes around the contained modules.

5.1 MAC FSM Granularity

REQ-037 defines the complete MAC frame field sequence and fixed-polarity bits for all in-scope formats. With one FSM state per MAC frame field, the FSM naturally progresses through frame structure: format-dependent transitions - such as the divergence between CC and CF at the FDF bit field - become state graph edges rather than counter conditionals inside a shared state. Each requirement pertaining to a specific MAC frame filed maps to a single FSM state aiding implementation, verification and traceability.

5.2 LLC Frame Format

The LLC layer defines two frame formats: a host-facing format that maintains compatibility with `can_bus_controller`, and a MAC-facing format designed to efficiently facilitate frame streaming.

5.2.1 Host-LLC Interface Format

The host-LLC format shown in Figure 10 extends the `can_bus_controller` LLC frame format, expanding the data field from 8 to 64 bytes and adding FDF, BRS, and ESI to the existing trailing control bytes alongside IDE and RTR. The field layout and byte positions are otherwise unchanged, so host software requires no change to the fields it already uses.

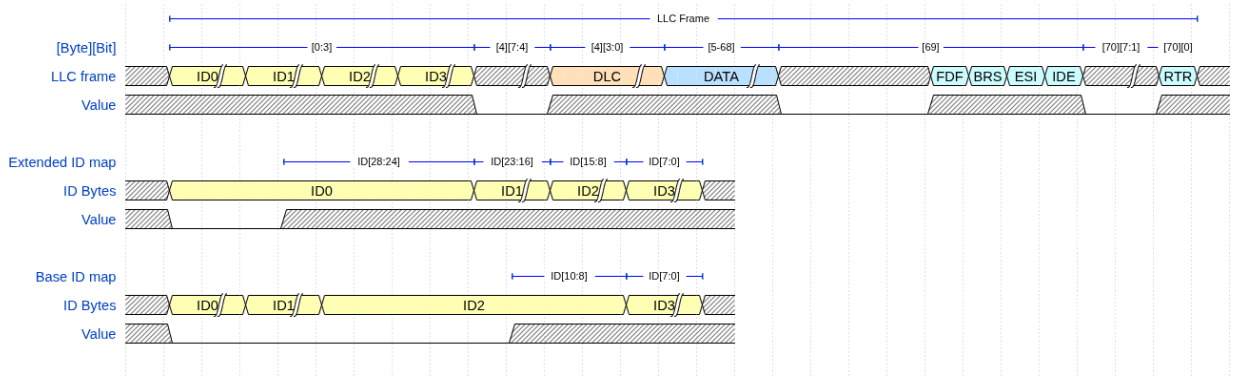

Figure 10: LLC frame format (71 bytes) at the host-LLC interface, with identifier byte mapping for base and extended IDs. Hatched regions indicate variable-value bits.

5.2.2 LLC-MAC Interface Format

The host-LLC format places all control flags after the payload, requiring a full 71-byte frame buffer before serialization can begin. The LLC-MAC format (Figure 11) front-loads all metadata into two leading config bytes, allowing `can_mac_ser` to begin streaming ID and data bytes immediately without buffering the full frame.

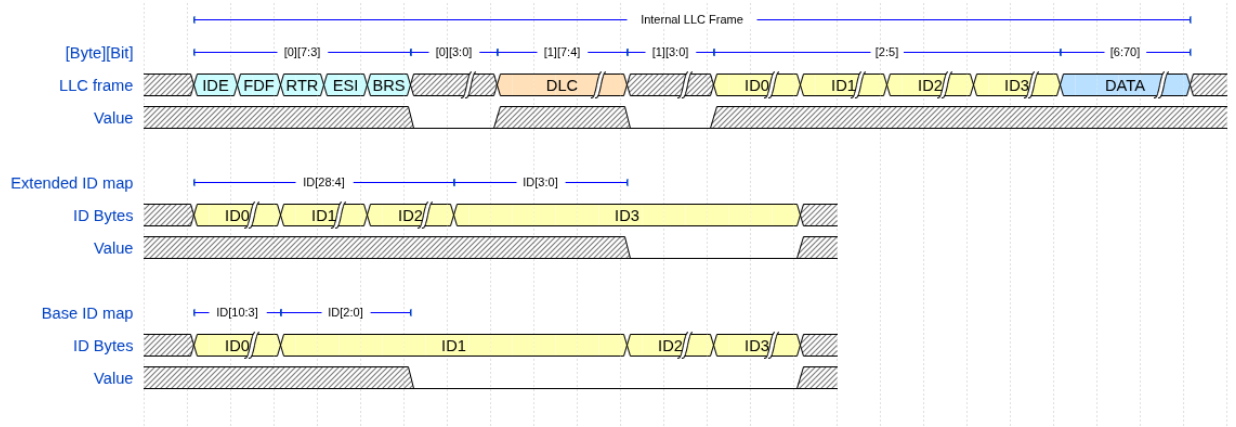

Figure 11: Internal LLC frame format at the `can_mac_ser` input, with identifier byte mapping for base and extended IDs. Hatched regions indicate variable-value bits.

6 Implementation

This section describes the RTL implementation of the modules introduced in Section 5, covering the key behavioral choices in each. All inter-module interfaces use typed records paired with reset constants, so every module can be reset without enumerating individual fields. Port direction follows `m2s/s2m` (master-to-slave/slave-to-master) for control interfaces and `s2d/d2s` for Avalon-ST data interfaces. `pk_can_types` is the single shared package defining every interface type, protocol constant, frame format layout, and

utility function used across the design. The complete signal-level schematic of `can_mac_pcs_fce` - the integrated implementation of the architecture overview in Section 5 - is reproduced in Section A.

`can_llc` was not implemented within the project timeline. Its interface contracts are fully specified in the verification plan (REQ-001 through REQ-005, REQ-031, REQ-034, REQ-036) and the implementation path is described in Section 9.2.

6.1 `can_mac_fsm`

`can_mac_fsm` orchestrates bit-level frame encoding and decoding across all four in-scope formats, coordinating `can_mac_bs`, `can_mac_crc`, and `can_pcs` on each sample-point cycle. The module accumulates received frames into a 70-byte internal byte array (`llc_frame`), reusing `can_bus_controller`'s register-array approach as a proof-of-concept baseline. The resource overhead is negligible at the 15 byte frame used by CC, but becomes the dominant logic element cost at the 70 byte frame used by CF - a block RAM migration is the identified upgrade path, see Section 9.2. Bus-off state is owned entirely by `can_fce` and `can_mac_fsm` just treats the `bus_off` signal from `can_fce` as a secondary reset alongside the hardware reset. ### FSM Structure

`can_mac_fsm` contains two synchronous processes:

- **`p_fsm`**: The main controlling FSM.
- **`p_stream_to_LLC`**: Responsible for streaming received frames to `can_llc` via Avalon-ST.

An `is_transmitter` flag, latched when `p_fsm` drives the SOF bit at the start of a new frame transmission and cleared at arbitration loss or at the end of the EOF field, partitions per-state logic into a TX branch and an RX branch. The error flag transmission states are an exception: both transmitter and receiver nodes enter the same two-state sequence, with flag polarity driven by the `error_active` signal from `can_fce`.

State transition are triggered by the `sample_point` signal from `can_pcs` and `p_fsm` organizes each sample-point cycle as three phases:

- **Pre-case code block**: This code block runs before the FSM case statement and handles all conditions that preempt normal state progression: stuff-bit insertion, lost arbitration, bit errors, and ACK errors on the TX branch, stuff-bit removal, stuff errors, form errors, CRC errors and overload conditions on the RX branch. When any of these conditions fire, the `v_skip_case` variable is set and the case block is bypassed entirely.
- **FSM case block**: This is the main FSM case statement. It handles only the error and stuff-bit free state transitions.
- **Post-case block**: This code block feeds the `can_mac_bs` and `can_mac_crc` and commits the drive polarity to the PCS output.

This structure trades per-state code locality for non-duplication of cross-cutting logic. Placing stuff-bit and error handling inside each state would duplicate identical detection and feed logic across multiple states. Centralizing it in the pre-case handles it once and the `v_skip_case` flag guarantees that the state machine does not advance when it should not. For logic that is genuinely per-state - DLC parsing, ACK success latching, EOF completion - the case block handles it directly.

`p_fsm` implements the per-field state granularity introduced in Section 5.1. Each post-arbitration field has a dedicated state, with the arbitration region sharing `s_arbitration` across ID bits, RTR/SRR/RRS, and IDE via `bit_count`. The complete `p_fsm` is shown in Figure 12.

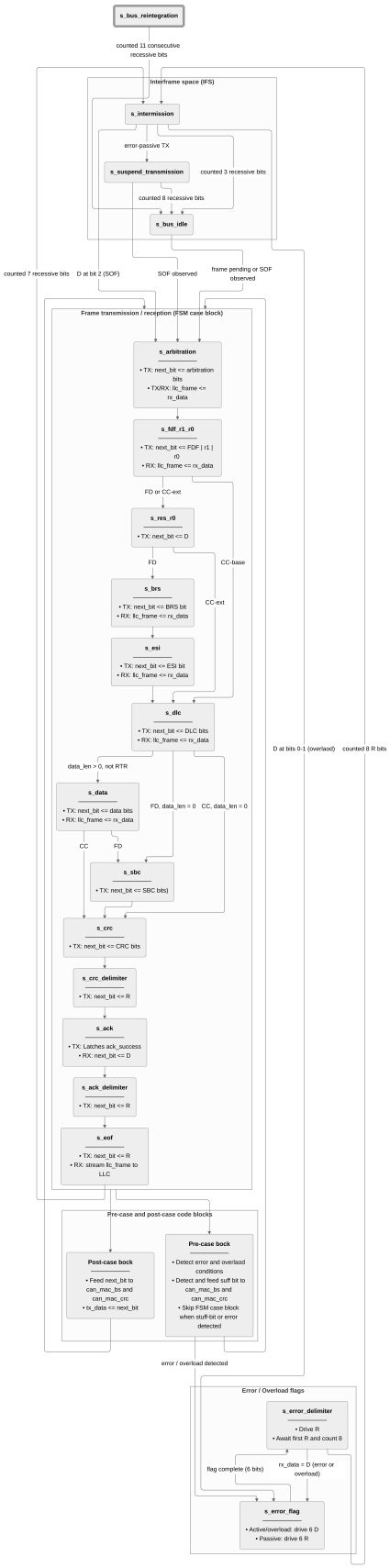

Figure 12: `can_mac_fsm`. FSM orchestrating frame transmission and reception in `can_mac`. The state machine encodes the four in-scope frame formats depicted in Figure 4. The states grouped in the FSM code block encode error and stuff-bit-free frame progression. Error and stuff-bit detection is handled in the pre-case code block. The next bit is driven to `can_pcs` and fed to `can_mac_bs` and `can_mac_crc` in the post-case code block.

6.1.1 TX Mode

The can_mac_fsm executes the TX-relevant logic when is_transmitter is set. The can_mac_fsm drives the next bit to can_pcs through the tx_data signal two clock cycles after each sample point - giving the can_mac_bs and can_mac_crc modules time to present valid outputs. This is necessary because the can_mac_bs output must be stable before the can_mac_fsm can decides whether a stuff bit is due, and can_mac_crc must hold the fully accumulated value before the first CRC bit is driven. The can_pcs module latches the tx_data signal at the bit boundary and can_mac_fsm simply needs valid data ready in time.

The can_mac_fsm samples the rx_data signal from can_pcs at each SP strobe from can_pcs - detecting bit error, ACK/ACK-error, and arbitration loss. In the CF data phase, the SSP strobe replaces the SP for bit-error monitoring. The can_mac_fsm compares rx_data against transmitted_bits_shift_reg, where tdc_delay (supplied alongside the SSP strobe by the can_pcs) is used to index into transmitted_bits_shift_reg. Arbitration loss clears is_transmitter in s_arbitration and the node continues as an receiver.

During the s_arbitration state multiple nodes may be transmitting. This necessitates both transmitters and receiver nodes to feed can_mac_bs and can_mac_crc from the rx_data signal. This ensures that transmitter and receiver roles have matching accumulators during arbitration - enabling seamless transmitter-to-receiver transitions on arbitration loss. From s_fdf_r1_r0 onward the transmitter nodes switches to transmitted_bits_shift_reg(tdc_delay) as the can_mac_bs and can_mac_crc feed source - enabling TDC in the data-phase.

6.1.2 RX Mode

The can_mac_fsm executes the RX-relevant logic when is_transmitter is not set. The can_mac_fsm observes the rx_data signal from can_pcs at each SP strobe and stores received bits directly into the llc_frame byte array. The can_mac_fsm feeds the can_mac_bs module from rx_data and uses the can_mac_bs valid output signal to trigger de-stuffing. can_mac_crc is also fed from rx_data and the relevant output is selected using the crc_poly_select select signal once the DLC field as been received. The can_mac_fsm validates the received SBC and CRC values against the locally accumulated result and checks form bit polarities and stuff errors. During the ACK slot the ACK bit is driven for one bit time. The CF ACK slot spans two bits, but the receiver asserts dominant only during the first. During the s_intermission state, the completed frame is streamed byte-by-byte to can_llc over the Avalon-ST interface by p_stream_to_LLC.

6.2 can_mac_ser

can_mac_ser converts the byte stream from can_llc into the serial bit stream consumed by can_mac_fsm. The module's FSM is comprised of four states, managing presentation of the frame metadata, byte fetching, and bit-by-bit serialization. can_mac_ser extracts the frame metadata (IDE, FDF, DLC, FTYP, BRS, ESI) from the two leading configuration bytes in the MAC-facing LLC frame (Section 5.2.2). These are bits are registered in t_llc_metadata which remains stable for the entire frame. can_mac_ser forwards the transfer_status signal from can_mac_fsm back to can_llc, returning to s_load_config_byte_0 reset state on any non-ongoing status. The padding bits in the ID field of the MAC-facing LLC frame are silently skipped, allowing can_mac_fsm to receive an uninterrupted stream of valid bits. The four-state can_mac_ser FSM is shown in Figure 13.

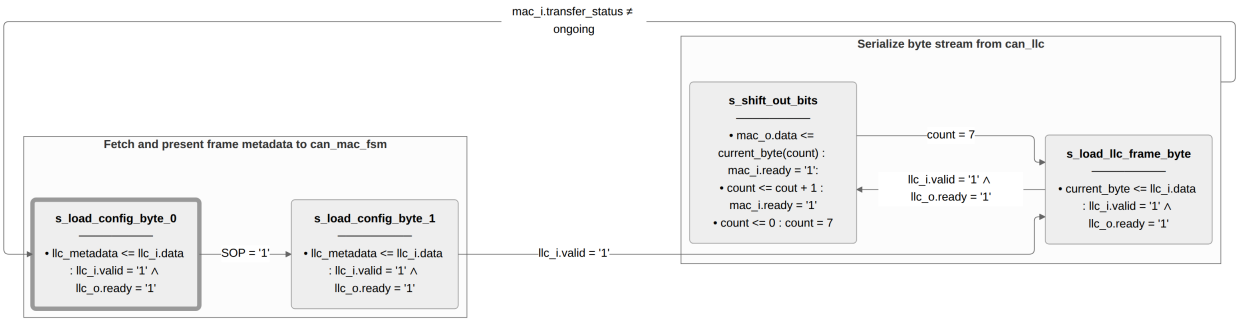

Figure 13: `can_mac_ser` FSM serializing the MAC-facing LLC frame (Section 5.2.2). The bit stream is presented to `can_mac_fsm` using a valid/ready handshake. Unused padding bits in the 32-bit ID field are skipped silently.

6.3 `can_mac_bs`

`can_mac_bs` implements both dynamic and fixed bit stuffing for CC and CF frames, see Figure 14. The entity is instantiated inside `can_mac` and serves both TX stuffing and RX de-stuffing. The module has a data/valid interface in both the input and output directions. `can_mac_fsm` drives the relevant bit stream through the input data/valid interface. The output data/valid interface is used by `can_mac_fsm` to either inserted stuff bits into the TX bit stream or discard them from the RX stream.

The module operates in dynamic bit stuffing mode when `can_mac_fsm` de-asserts the `fixed_bit_stuffing_en` input signal. In this mode, an inverse-polarity stuff bit is emitted after after every five same-polarity bits received from `can_mac_fsm`. A counter is incremented on each emitted stuff bit and the value of this counter is Gray-coded and parity bit is added, generating the `stuff_bit_count` output signal constituting the SBC field in the CF MAC frame format. When operating in fixed mode (`fixed_bit_stuffing_en` asserted), the module emits a fixed stuff bit immediately on the rising edge of `fixed_bit_stuffing_en`, then one fixed stuff every four bits.

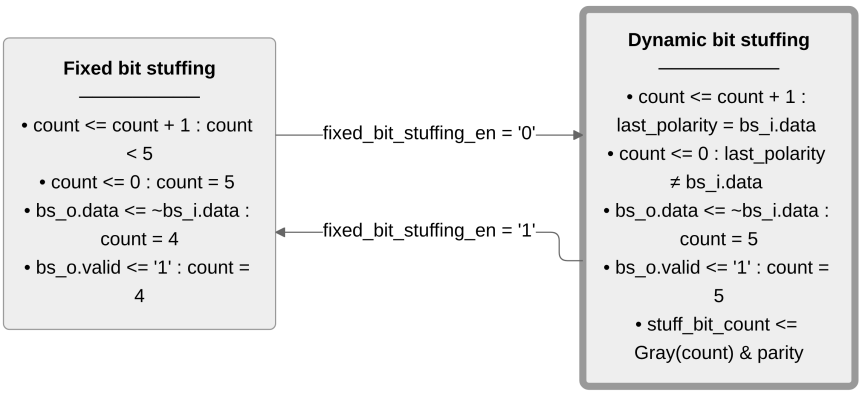

Figure 14: `can_mac_bs` bit stuffing logic.

6.4 `can_mac_crc`

`can_mac_crc` instantiates three parallel `gen_crc` modules (`u_crc15`, `u_crc17`, and `u_crc21`) with their output signals routed through a multiplexer using the `crc_poly_select` input signal as select, see Figure 15. For CC frames, the `can_mac_fsm` drives the input data through the `data_cc`/`valid_cc` interface to `u_crc15`. For CF frames, input data is driven through the `data_fd`/`valid_fd` interface feeding the `u_crc17` and `u_crc21` entities.

The multiplexer output is left-aligns it to the common 21-bit output width. The output of `u_crc15` occupies bits [20:6], `u_crc17` occupies bits [20:4], and `u_crc21` occupies the full width. Transmitter nodes set `crc_poly_select` from the DLC field in `llc_metadata` before the first frame bit is driven. Receiving nodes set `crc_poly_select` after the DLC field has been received.

The output multiplexer is kept combinatorial - each `gen_crc` instance registers its outputs, so the multiplexer already selects over registered values. This allows `can_mac` to meet the IPT ≤ 2 TQ requirement at the minimum pre-scaler value (m=1 => TQ = 1 system clock period) (REQ-024).

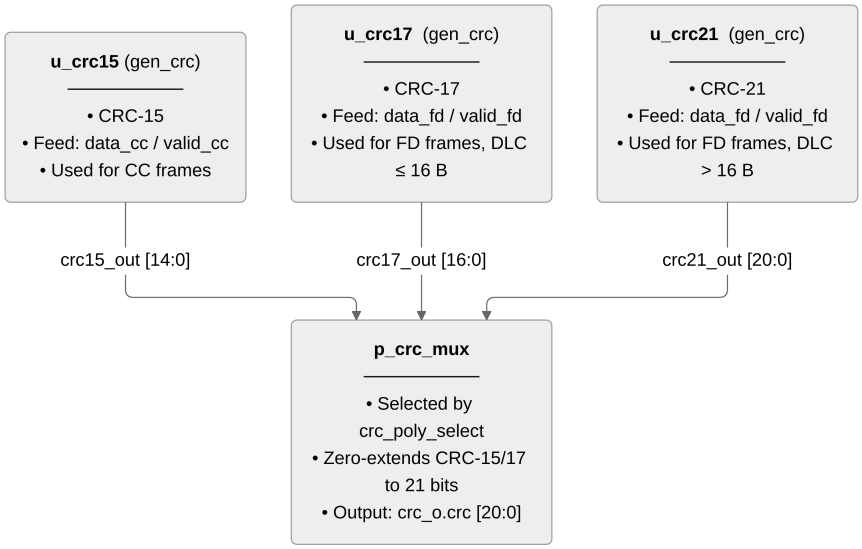

Figure 15: `can_mac_crc` data flow with three parallel `gen_crc` engines. The output multiplexer is combinatorial to satisfy REQ-024 (IPT ≤ 2 TQ) at minimum prescaler configuration.

6.5 `can_fce`

`can_fce` drives the `error_active` and `bus_off` signals controlling the error state of `can_mac`. `can_fce` maintains TEC and REC and counter update logic follow REQ-028. The logic is implemented through the 3-state FSM depicted in Figure 16.

Bus-off recovery requires 128 strobes of `idle_condition` signal from the `can_pcs` module, after which both TEC and REC reset to zero and the FSM returns to the `s_error_active` reset state. Reset to `s_error_active` can can also be initiated through the `normal_mode` signal from `can_llc`.

The TEC incrementation exception specified in REQ-028 is implemented through the `passive_tx_ack_error_exempt_` signal asserted by `can_mac_fsm` when detecting ACK error while `error_active` is de-asserted.

6.6 `can_pcs`

`can_pcs` implements bit timing as a 4-state FSM, mapping to the four CAN bit-time segments described in Section 3.3, see Figure 17. The module is parameterized through a set of generics specifying the segment lengths, SJW and prescaler values. The FSM advances through `s_sync_seg` (1 TQ, fixed), `s_prop_seg`, `s_phase_seg1`, and `s_phase_seg2` as each segment's TQ count expires. At the end of the `s_phase_seg1` state, the `sample_point` strobe signal fires and the transceiver RX signal is latched to the `rx_data` - both consumed by `can_mac_fsm`. At the end of the `s_phase_seg2` state, the `tx_data` input signal from `can_mac_fsm` is latched to the transceiver TX signal. When the `bus_off` input signal is asserted by

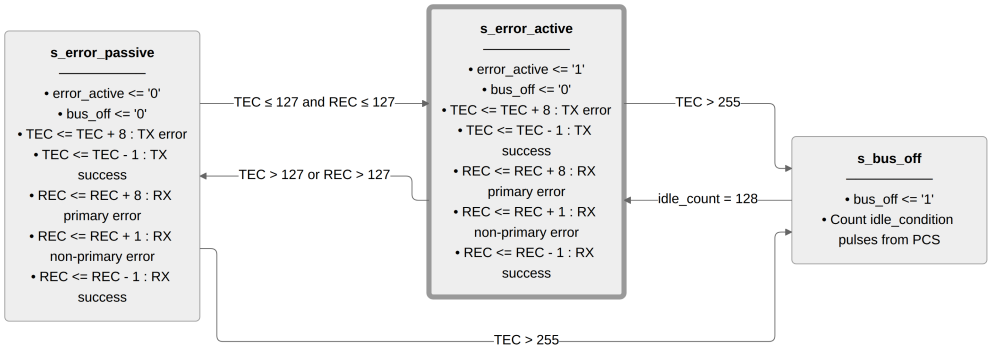

Figure 16: `can_fce` FSM governing the error active, error passive, and bus off node states.

`can_fce`, all output signals to `can_mac_fsm` are suppressed and the `idle_condition` signal (consumed by `can_fce`) is pulsed every 11 consecutive R bits.

Hard synchronization is initiated by the `can_mac_fsm` through the `do_hard_sync` input signal. When this signal is asserted, the module performs hard synchronization on the first observed R-to-D edge. On other R-to-D edges, the module performs re-synchronization by adjusting the `s_phase_seg1` and `s_phase_seg2` state durations by SJW. Both hard and re-synchronization are only enabled when the CAN node is operating as receiver, indicated by `can_mac_fsm` through the `transmitting` input signal.

6.6.1 Dual Bit Rate Switching

`can_pcs` holds no frame-format knowledge. Rate switching is entirely driven by `can_mac_fsm` through three dedicated control signals.

- `next_bit_is_brs`: When this signal is asserted and the bus is observed or driven R at the sample point, the module replaces the nominal segment lengths, SJW and prescaler values with the data-phase counterparts.
- `next_bit_is_res`: When this signal is asserted at the end of the `s_phase_seg2` state, the module starts TDC measurement.
- `data_phase_stop`: When this signal is asserted and the bus is observed or driven R at the sample point, the module switches back to nominal bit-timing parameters. The signal is asserted by `can_mac_fsm` at the CRC delimiter bit sample point or when an error is observed.

6.6.2 Transmitter Delay Compensation

TDC is implemented as a three-stage pipeline, active in nodes operating as transmitters:

1. **Measure**: The measurement starts when the D polarity CF res bit is driven to the transceiver TX output signal. The `delay_count_tq` counter increments each TQ until the transmitted D polarity edge is observed on the transceiver RX input signal. When measurement is complete the `tdc_delay` output signal is latched
2. **Count down**: When the first data-phase bit is transmitted, the previously measured `delay_count_tq` delay is counted down each TQ. When reaching zero, the `ssp_active` flag is asserted and the `tdc_delay` output signal is latched (used by `can_mac_fsm` to index into the transmitted-bit shift register enabling bit error detection during the CF data-phase).
3. **Fire**: With `ssp_active` asserted, the `secondary_sample_point` output strobe fires one TQ before the `sample_point` strobe on every data-phase bit time. `can_mac_fsm` uses `tdc_delay` to index into

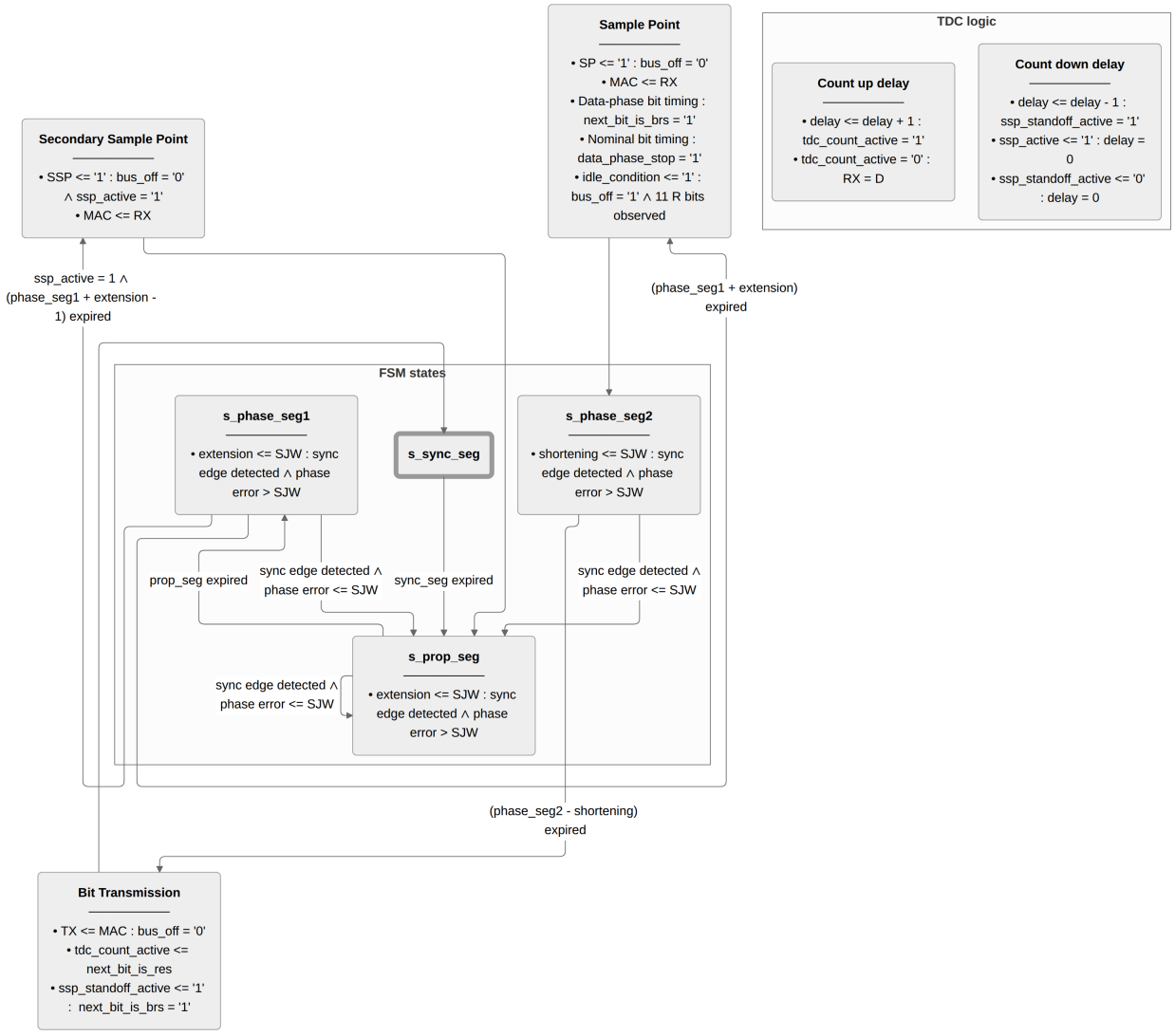

Figure 17: `can_pcs` FSM implementing CAN node bit timing, synchronization and TDC logic.

the transmitted-bit shift register for bit error detection.

7 Verification and Results

The implemented modules described in Section 6 were exercised against the 37-requirement verification plan using five unit testbenches and one integration level testbench (`can_mac_pcs_fce_tb`), see Figure 18. The majority of requirements are closed by simulation. Code inspection provides evidence where testbench stimulus cannot reach the required condition. REQ-023 (overload frame conditions) is the primary example: triggering an overload frame requires injecting a dominant bit at specific field boundaries, which requires a frame-aware bit injector not present in the current testbench.

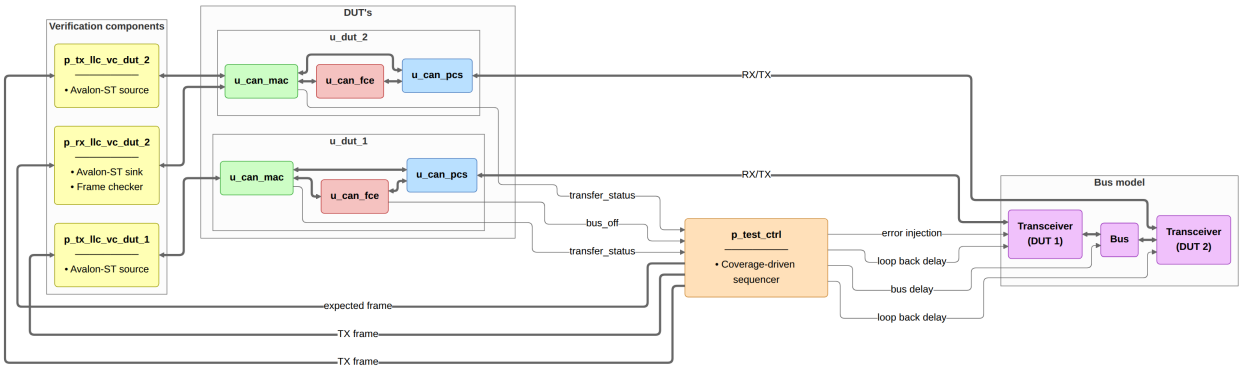

Figure 18: `can_mac_pcs_fce_tb` integration testbench. Two `can_mac_pcs_fce` instances connect through a dominant-wins bus model. Avalon-ST Verification Components (VCs) drive and sample the MAC interfaces. `p_test_ctrl` sequences test stimuli, injects bit errors, reads transfer status, and monitors bus-off status.

Code inspection provides evidence for sub-claims in several requirements covered in Section 7.2:

- **Error detection paths not exercised by simulation**: REQ-021 sub-claims 2-5 (stuff, form, CRC, and ACK error detection) and REQ-022 sub-claims 2-3 (data-phase error rate switching and CRC error receiver behavior) are verified by code inspection only - frame-aware error injection is not available in the current testbench.
- **Receiver acceptance of non-standard bit values**: REQ-012 sub-claims 1-2 (receiver acceptance of dominant SRR and RRS bits without form error) are confirmed by inspecting the form-error logic in `can_mac_fsm.vhd`.
- **Conditions not directly observable in simulation**: REQ-010 sub-claim 2 (third-intermission-bit skip-SOF transition) is confirmed by reading the guard conditions in `can_mac_fsm.vhd`.

7.1 Unit Testbench Simulation

The five unit testbenches target individual submodules with focused stimulus.

1. **`can_mac_crc_tb`**: closes REQ-006 via three coverage bins (CRC-15 for CB/CE, CRC-17 for FD ≤16 bytes, CRC-21 for FD >16 bytes). The testbench contains an independent software CRC reference model (`f_calc_can_crc`) with no shared logic with the DUT: it processes the bit stream serially and applies the ISO polynomial to produce the expected digest, checked against DUT output on every frame. Additional checkers verify init vector reset and output stability between frames.
2. **`can_mac_bs_tb`**: closes REQ-016 and REQ-018 via a two-phase stimulus: six directed FSB tests followed by random dynamic bits. Three concurrent reference models verify DUT out-

put: `p_stuff_bit_checker` verifies a complement bit after every five same-polarity bits, `p_sbc_checker` verifies Gray-code and parity bit. All coverage bins are hit.

3. **`can_pcs_tb`**: provides simulation evidence for REQ-024, REQ-025, REQ-026 (combined with code inspection), and REQ-027. Two `can_pcs` instances run on mismatched clocks through a physical bus model. Three tests cover reset, random CF frames with alternating clock leadership, and bus-off isolation. `p_check_tdc_delay` verifies that `polarity_history(tdc_delay)` matches the sampled bus value at each SSP strobe. `p_rx_mac_vc` compares RX-sampled bits against the TX sequence bit by bit.
4. **`can_fce_tb`**: closes REQ-028, REQ-029, and REQ-033 (sub-claim 2) using directed stimulus. Reset confirms outputs clear and `llc_i.normal_mode` restores error-active from any state. All REQ-028 counter rules are exercised at the error-active/error-passive boundary. Asserting `passive_tx_ack_error_exempt_1` while error-passive confirms no bus-off entry (REQ-033 sub-claim 2). 64 `idle_condition` pulses confirm no premature release and 128 confirm clearance and error-active restoration (REQ-029).
5. **`can_mac_ser_tb`**: closes REQ-031 (DLC encoding: linear for DLC 0-8, FD non-linear for DLC 9-15) and provides supporting evidence for higher-level requirements. Three coverage dimensions drive frame selection: IDE (base/extended), FDF (CC/FD), and DLC (0-15). `p_mac_fsm_vc` verifies all metadata fields against the LLC frame and compares the output bit stream byte by byte, accounting for ID width, padding, and DLC-derived data length. Random back pressure and a 2% mid-frame abort rate exercise pipeline stalls and the `c_disturbed` path. `p_transfer_status_checker` monitors `transfer_status` every cycle.

7.2 Integration Testbench Simulation

`can_mac_pcs_fce_tb` is the primary integration level testbench, exercising two `can_mac_pcs_fce` instances connected through a dominant-wins bus model and covering 17 requirements spanning MAC frame encoding, arbitration, and error handling. The following waveforms are selected excerpts from the test run. Many requirements are verified by waveform inspection across all test scenarios. Reproducing a dedicated figure for each requirement would be impractical, so only representative scenarios are shown.

- **Complete FD Frame Transmission** (Figure 19): A complete FD frame transmitted by DUT 1 and received by DUT 2, showing hard synchronization on SOF, TDC loopback delay measurement, dual bit rate switching at the BRS sample point, SSP pulses during the data phase, and ACK confirmation (REQ-010, REQ-012, REQ-014, REQ-015, REQ-017, REQ-026).
- **Bit Stuffing** (Figure 20): Dynamic and fixed mode behavior (REQ-016, REQ-018).
- **Bit Rate Switching and TDC** (Figure 21): The PCS switches to data-phase bit timing at the BRS sample point and positions the SSP once the transceiver loopback delay is measured (REQ-025, REQ-030).
- **Arbitration** (Figure 22): The losing node clears `is_transmitter` in-place at `s_arbitration` and continues as receiver without a state transition (REQ-020).
- **Error Handling and Bus-off Recovery** (Figure 23, Figure 24): Error frame escalation and bus-off recovery (REQ-007, REQ-008, REQ-009, REQ-021, REQ-022, REQ-028, REQ-029, REQ-033 sub-claim 1).

7.3 Testbench Results Summary

Of the 37 requirements, 28 are closed: 26 via testbench simulation (Table 6) and two (REQ-013, REQ-023) via code inspection. Nine remain open: six are LLC requirements (REQ-001 through REQ-005, REQ-036) deferred pending `can_llc` implementation, REQ-034 (P3) and REQ-035 (P2) are non-blocking, and REQ-

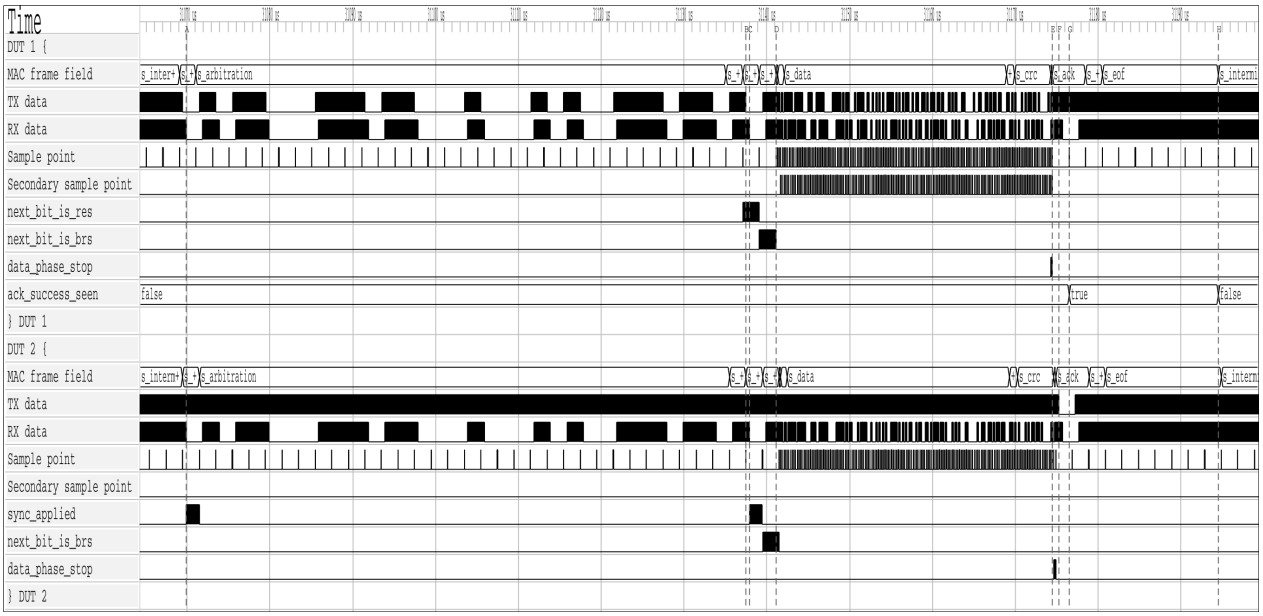

Figure 19: Two-node simulation of a complete FD frame transmission in `can_mac_pcs_fce_tb`. DUT 2 hard-synchronizes on SOF at A. DUT 1 measures the TDC loopback delay between B and C. Both nodes switch to data-phase bit timing at the BRS sample point at D. DUT 1 switches back to nominal bit timing at the CRC delimiter sample point at E. DUT 2 drives the ACK slot dominant at F, DUT 1 samples the dominant ACK at G and latches `ack_success_seen`. Transmission ends at H.

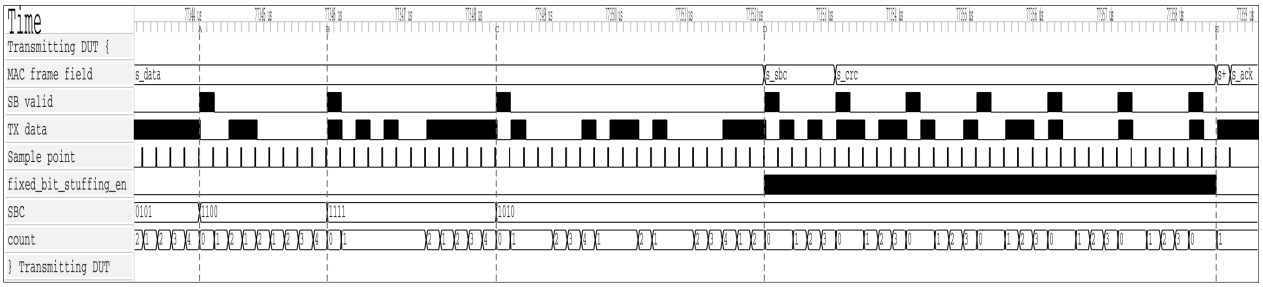

Figure 20: Dynamic and fixed bit stuffing in `can_mac_pcs_fce_tb`. Dynamic stuff bits are inserted at A, B, and C in the `s_data` region. At D, `fixed_bit_stuffing_en` asserts and the stuffer switches to fixed mode for `s_sbc` and `s_crc`. Seven fixed stuff bits are inserted between D and E.

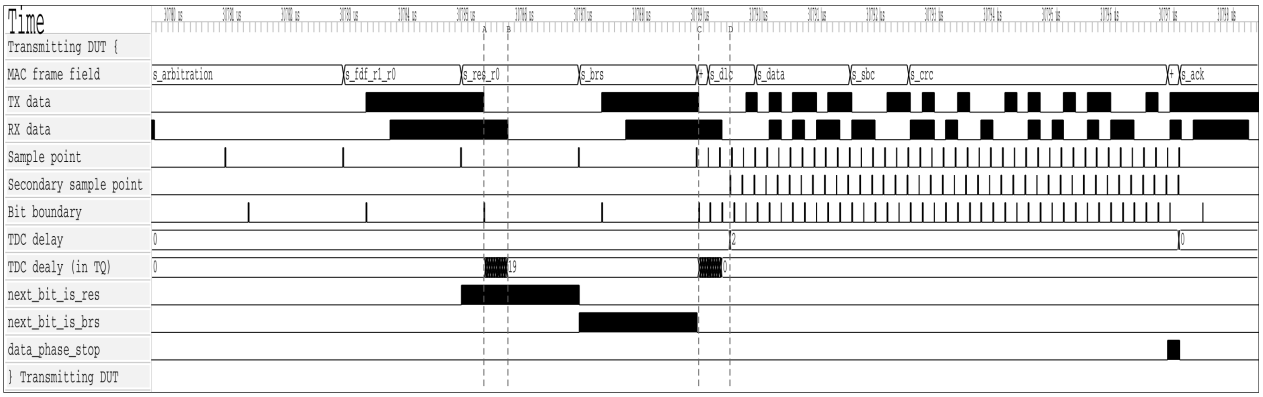

Figure 21: Dual bit rate switching and TDC measurement in `can_mac_pcs_fce_tb`. At A, the PCS begins counting the transceiver loopback delay in TQ increments. At B, the transmitted bit arrives on RX and the count stops at 19 TQ. At C, the first data-phase bit (ESI) is transmitted and the measured delay is counted down. When the countdown terminates at D, the SSP strobe activates and the TDC delay of 2 TQ is reported to the MAC. `next_bit_is_res` and `next_bit_is_brs` mark the measurement window boundaries. `data_phase_stop` signals the end of the data phase.

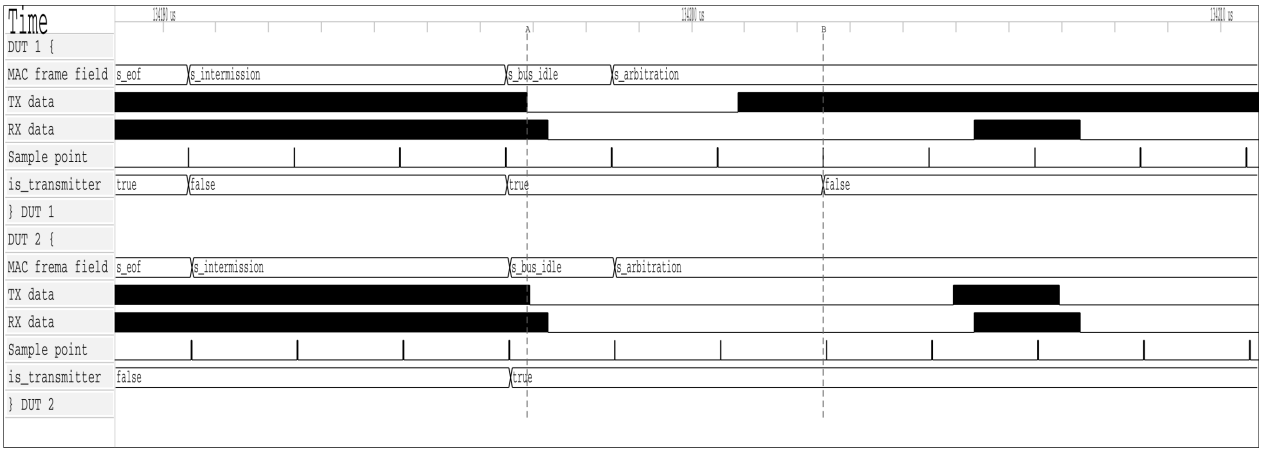

Figure 22: Arbitration loss in `can_mac_pcs_fce_tb`. Both nodes enter `s_arbitration` as transmitters at A. DUT 1 loses arbitration at B after transmitting recessive and sampling dominant, and continues as receiver with `is_transmitter` cleared.

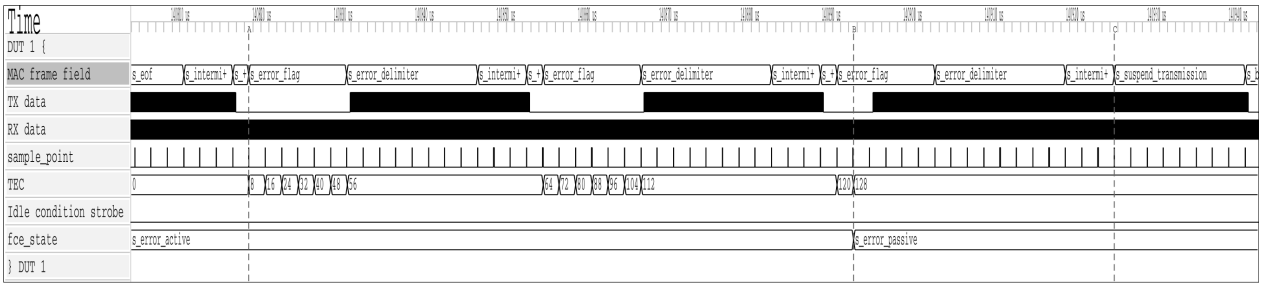

Figure 23: Error frame escalation in `can_mac_pcs_fce_tb`. A bit error on the SOF bit triggers the first error flag at A, incrementing TEC to 8. Each dominant bit of the error flag is sampled as a bit error because the bus is held recessive, rapidly escalating TEC. At B, TEC reaches 128 and `fce_state` transitions to `s_error_passive`. At C, the node enters `s_suspend_transmission` after `s_intermission`.

Figure 24: Bus-off recovery in `can_mac_pcs_fce_tb`. DUT 1 transmits the first active error flag at A. At B, TEC reaches 128 and the node transitions to error passive. At C, TEC reaches 256 and the node enters bus off. The FCE counts 128 idle condition strobes from the PCS and restores `s_error_active` at D. At E and F, DUT 2 acknowledges the first two frames transmitted after recovery.

021 (P1) is partially covered - bit-error detection is verified in `test_bus_off`, while the remaining sub-claims (stuff, form, CRC, ACK) require frame-aware error injection not available in the current testbench (Section 9.2).

Table 6: Testbench execution status and requirements coverage.

Testbench	Requirements covered	Status
`can_mac_crc_tb`	REQ-006	Pass
`can_mac_bs_tb`	REQ-016, REQ-018	Pass
`can_pcs_tb`	REQ-024, REQ-025, REQ-026, REQ-027	Pass
`can_fce_tb`	REQ-028, REQ-029, REQ-033 (sub-claim 2)	Pass
`can_mac_ser_tb`	REQ-031	Pass
`can_mac_pcs_fce_tb`	REQ-007, REQ-008, REQ-009, REQ-010, REQ-011, REQ-012, REQ-014, REQ-015, REQ-017, REQ-019, REQ-020, REQ-021 (bit error only), REQ-022, REQ-030, REQ-031, REQ-032, REQ-033 (sub-claim 1)	Pass

8 Synthesis

The implemented `can_mac_pcs_fce` stack was synthesized targeting the Cyclone 10 LP device used in Everllence's IO-extender board. The clock constraint was set to 6 ns (166 MHz), deliberately overconstraining relative to any realistic CAN FD system clock to expose worst-case timing paths.

8.1 Resource Utilization

Table 7 compares resource utilization by function between `can_bus_controller` (CAN CC) and `can_mac_pcs_fce` (CAN FD) on the Cyclone 10 LP target. The FD stack uses 4,608 Logic Elements at 30% device utilization against 1,146 for the CC baseline.

Table 7: Resource utilization on Cyclone 10 LP by function: `can_bus_controller` (CC) vs `can_mac_pcs_fce` (CF).

Function	`can_bus_controller`	LEs	Regs	`can_mac_pcs_fce`	LEs	Regs
Protocol FSM	`can_fsm`	589	86	`can_mac_fsm`	4,109	684
Bit timing	`can_node_clock`	110	25	`can_pcs`	190	49
Fault confinement	*(in `can_fsm`)*	-	-	`can_fce`	117	31
CRC	`gen_crc`	18	15	`can_mac_crc`	84	53
TX serializer	`can_ast_to_serial`	92	34	`can_mac_ser`	84	40
RX frame buffer	`can_serial_to_ast`	324	166	*(in `can_mac_fsm`)*	-	-
Bit stuffing	`can_stuff_bit_gen`	10	6	`can_mac_bs`	31	12
Total		**1,146**	**334**		**4,608**	**869**

`can_mac_fsm` accounts for 89% of CF LEs. In the CC design the RX frame buffer is a separate module (`can_serial_to_ast`, 324 LEs). In the CF design it is integrated into `can_mac_fsm`, which combines protocol FSM and frame buffer in a single entity. The LE growth analysis is in Section 9.

8.2 Timing Results

Timing analysis covers four PVT corners: slow/fast transistor speeds, 1150 mV supply voltage, and -40°C to 100°C temperature range. Table 8 shows the setup and hold slack at the overconstraining 166 MHz clock.

Table 8: Timing results for `can_mac_pcs_fce` at 6 ns (166 MHz) on Cyclone 10 LP. Slow/Fast denotes process transistor speed.

Corner	Worst setup slack	fmax estimate	Hold slack
Slow 1150 mV 100°C	−1.858 ns	~127 MHz	+0.236 ns
Slow 1150 mV −40°C	−1.550 ns	~152 MHz	+0.021 ns
Fast 1150 mV 100°C	+0.230 ns	>166 MHz	+0.163 ns
Fast 1150 mV −40°C	+0.580 ns	>166 MHz	+0.145 ns

The worst-case fmax is approximately 127 MHz on the slow 100°C corner. The negative setup slack on slow corners indicates timing violations at the 166 MHz constraint - a consequence of the deliberate overconstrain. Practical CAN FD implementations use system clocks of 20, 40, or 80 MHz [6]. The 127 MHz fmax exceeds the highest of these by more than 1.5×, meeting timing on all corners at any realistic operating frequency. Hold slack is positive across all corners.

9 Discussion

The three design-facing verification plan dimensions shaped the implementation in distinct ways. `layer` produced the module-to-testbench decomposition: each requirement maps to one testable module, and reference models in the sub-module testbenches made white-box sub-claims tractable without exposing RTL internals. `format_applicability` motivated front-loaded config byte ordering and per-field FSM granularity. `side` encoded requirements along the ISO transmitter/receiver axis and scopes testbench stimulus to the corresponding drive mode.

The AI-assisted workflow delivered value in two distinct phases, with very different characteristics in each. In the extraction phase, the LLM agent earned its keep by bootstrapping and linking the initial normative statement set. Having a fully populated and linked starting point - even one requiring substantial revision - gave the manual review process a concrete artifact to work from. The time saving from the extraction itself was, however, marginal. The agent's output had to be reviewed statement by statement, which is functionally similar to extracting requirements manually in the first place. The primary benefit of the AI-assisted approach in this phase was therefore not efficiency, but rather the increased consistency of an automated pass over the full standard text.

The MCP server interface proved genuinely useful throughout the design, implementation, and verification phases that followed extraction. As implementation decisions were made, requirements were refined - paraphrases sharpened, notes extended, observability classifications updated, and traceability fields populated. All updates were applied using the AI agent through dedicated schema-validated MCP tool calls, each targeting an individual requirement field. The narrow, validated interface made incremental AI-assisted maintenance of the verification plan safe and practical across all three project phases.

Of the nine open requirements, six are LLC requirements deferred pending `can_llc` implementation - a known scope boundary, not a gap in the implemented protocol engine. REQ-034 (P3) is not applicable: this implementation streams frames from LLC to MAC with no shared memory, so the shared memory

consistency requirement does not apply. REQ-035 (P2) is an optional operational feature - a node that always signals errors is a fully correct CAN FD participant. REQ-021 (P1) is partially covered: bit-error detection is verified, but sub-claims 2-5 require a frame-aware stimulus source to inject errors at the correct bit positions. A reference model approach closes them (Section 9.2).

The CAN FD stack uses 4,608 logic elements on the Cyclone 10 LP target - ~4 × the 1,146 elements of the existing CC controller `can_bus_controller`. Growth is dominated by the RX frame buffer in `can_mac_fsm`. The buffer spans 70 bytes (560 flip-flops), of which the 64-byte data payload contributes 512. The register growth tracks the payload increase directly: payload flip-flops grow from 64 (8 bytes) to 512 (64 bytes), accounting for 448 of the 535 additional registers over the CC baseline. Each data flip-flop at position `[byte_index=N][bit_index=M]` requires an individual write-enable signal:

```
WE = (state = s_data) AND sample_point AND (byte_index = N) AND (bit_index = M)
```

On a 4-input LUT, evaluating this condition without logic sharing costs 4 LUTs per flip-flop: 2 for the 6-bit `byte_index` comparator, 1 for the 3-bit `bit_index` comparator, and 1 to AND with state and sample point. Accordingly, 64 bytes × 8 bits × 4 LUTs = 2,048 LEs are needed to implement write enable across the 64-byte payload.

The `p_stream_to_LLC` process, which streams the received 70-byte frame byte-by-byte to `can_llc`, iterates through the received frame using a runtime counter. This synthesizes to a 70:1 8-bit mux. A 4-input LUT can implement a 2:1 mux, with 70 - 1 = 69 2:1 muxes needed for each output bit. This gives a total of 69 × 8 = 552 LEs for the full 70:1 8-bit mux. Table 9 summarises the resulting upper bound.

Together these bound the buffer contribution at approximately 2,716 LEs, leaving a minimum of 1,393 LEs for protocol FSM logic - at least 2.4× the 589 LEs of the CC `can_fsm`. However, the CC `can_fsm` includes partial fault confinement logic that in the CF design is separated into `can_fce` (117 LEs), so the 589 LE denominator is inflated relative to pure frame TX/RX logic. Subtracting the full `can_fce` cost as an upper bound gives a corrected CC baseline of 472 LEs, raising the protocol logic growth to at most 3.0×. The true ratio lies in the range 2.4× to 3.0×. The exact buffer/protocol split could be resolved by a controlled synthesis experiment, replacing `llc_frame` with constants and measuring the resulting LE reduction directly.

Migrating the 70-byte buffer to a single block RAM instance would eliminate both the flip-flops and decode logic, reducing `can_mac_fsm` to protocol-logic cost (Section 9.2). The worst-case fmax of approximately 127 MHz exceeds the highest recommended CAN FD system clock of 80 MHz by more than 1.5× (Section 8.2).

Table 9: `can_mac_fsm` LE upper-bound model. FFs in the protocol residual row are non-buffer registers (state, counters, control signals).

Component	FFs	LEs (upper bound)
Data payload write decode (64 bytes × 8 bits)	512	2,048
ID write decode (4 bytes × 8 bits)	32	96
Config writes (fixed address)	16	20
Streaming read mux (70:1 × 8 bits)	-	552
Frame buffer total	**560**	**~2,716**
Protocol FSM logic (residual)	124	≥1,393
`can_mac_fsm` total (measured)	**684**	**4,109**

9.1 Objectives Assessment

The three objectives stated in Section 1.4 are assessed against the verification results.

1. **CAN/CAN FD protocol controller in VHDL compliant with ISO 11898-1.** The unified `can_mac_pcs_fce` handles all four in-scope frame formats (CB, CE, FB, FE) in both TX and RX, including dual bit rate switching with TDC (REQ-024, REQ-025). The implementation covers the MAC, PCS, and FCE sub-layers. The LLC sub-layer is deferred as the one unimplemented module, leaving six LLC requirements open. 28 of 37 requirements are closed. The nine open cases are documented in Section 9.2.

2. **Structured requirements with traceability from ISO 11898-1 to testbench results.** 37 requirements were derived from ISO 11898-1, each linked to its source section, verification method, testbench file, and assertion label. 28 are closed against passing testbenches or code inspection. The full plan is in Section B.

3. **RTL design integrated via Avalon-ST interfaces into Everllence's existing FPGA infrastructure.** The RTL is written in portable VHDL-93. Synthesis confirmed a worst-case fmax of 127 MHz, exceeding the highest recommended CAN FD system clock by more than 1.5× at 30% device utilization (Section 8). `can_mac_pcs_fce` exposes a fully functional Avalon-ST TX/RX interface and can be driven directly by the host, following the same integration model as `can_bus_controller`.

9.2 Future Work

1. **`can_llc` implementation.** The LLC sub-layer is the one unimplemented module. Interface contracts are specified in REQ-001 through REQ-005 and REQ-036. Implementing it adds ISO-specified frame buffering and retransmission, completing the full ISO 11898-1 CAN node.

2. **Hardware integration and bring-up.** The RTL has been verified in simulation only. Bring-up on a physical CAN FD bus would validate timing closure, transceiver compatibility, and bit timing calibration under real bus conditions.

3. **CAN XL support.** CAN XL is out of scope. Extending the implementation to support CAN XL is a natural next step, building on the layered architecture established here.

4. **Frame buffer block RAM migration.** Migrating the 70-byte RX frame buffer to a single M9K block RAM instance would reduce `can_mac_fsm` from 4,109 LEs to protocol-logic-only cost. See Section 9 for the analysis.

5. **Error-type-specific simulation coverage (REQ-021).** Sub-claims 2-5 require a frame-aware reference model running in parallel with the integrated testbench. The model must track bit positions within the bus bit stream to identify when the bus is carrying a specific frame field. Stimulus can then be injected at the correct position to produce a targeted bit error, stuff error, form error, or CRC error, and the FSM response verified against the expected error handling path. This is most naturally implemented as an OSVVM verification component that shadows the FSM state and asserts injection commands at the right cycle.

10 Conclusion

This thesis presented the design, partial implementation, and verification of a CAN/CAN FD protocol controller in VHDL-93, following the ISO 11898-1 layered reference model. The implemented design covers

the MAC, PCS, and FCE sub-layers as independently testable modules, supports all four in-scope frame formats (CB, CE, FB, FE), and implements dual bit rate switching with Transmitter Delay Compensation. 28 of 37 requirements are closed against passing testbenches or code inspection. The nine open cases are documented in Section 9.2. The design uses 4,608 logic elements - 4.0× the CC baseline - with a worst-case fmax of approximately 127 MHz, exceeding the highest recommended CAN FD system clock of 80 MHz by more than 1.5×. Analysis attributes the majority of this growth to the RX frame buffer write-enable decode and streaming read mux, with protocol FSM logic growing 2.4× to 3.0× over the CC equivalent - consistent with the added protocol features. Migrating the frame buffer to block RAM is identified as the primary LE reduction path. The ISO 11898-1 layered reference model proved to be a practical partitioning of protocol complexity: each sub-layer was implemented, verified, and debugged independently, a property that motivates Everllence's decision to develop the controller in-house.

11 References

[1] Bosch, "CAN specification version 2.0," Bosch, 1991.

[2] F. Hartwich, "CAN with flexible data-rate," in *iCC 2012*, 2012.

[3] International Organization for Standardization, "Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer," ISO 11898-1:2024, 2024.

[4] J. Charzinski, "Performance of the error detection mechanisms in CAN," in *iCC 1994*, 1994.

[5] Texas Instruments, "TCAN1042-Q1 automotive CAN FD transceiver." 2024. Available: https://www.ti.com/lit/ds/symlink/tcan1042-q1.pdf

[6] A. Mutter, "Robustness of a CAN FD bus system — about oscillator tolerance and edge deviations," in *iCC 2013*, 2013.

[7] A. Mutter and F. Hartwich, "Advantages of CAN FD error detection mechanisms compared to classical CAN," in *iCC 2015*, 2015.

[8] M. Jeřábek and P. Píša, "CTU CAN FD." 2019. Available: https://github.com/Logic-Design-Services/CTU-CAN-FD

[9] International Organization for Standardization, "Road vehicles — controller area network (CAN) conformance test plan — Part 1: Data link layer and physical signalling," ISO 16845-1:2016, 2016.

[10] Bosch, 2024. Available:https://www.bosch-semiconductors.com/products/ip-modules/can-ip-modules/m-can/

[11] AMD, 2024. Available: https://www.amd.com/en.html

[12] CAST, Inc., 2024. Available: https://www.cast-inc.com/interfaces/automotive-bus-controllers/can-ctrl

[13] Intel Corporation, "Quartus Prime." 2024. Available:https://www.altera.com/products/development-tools/quartus

[14] OSVVM, 2024. Available: https://osvvm.org

[15] Aldec, Inc., "Riviera-PRO." 2024. Available: https://www.aldec.com/en/products/functional_verification/riviera-pro

[16] Sigasi, "Sigasi Studio." 2024. Available: https://www.sigasi.com

[17] A. Bybell, "GTKWave." 2024. Available: https://gtkwave.sourceforge.net

[18] WaveDrom, 2024. Available: https://wavedrom.com

[19] Mermaid, 2024. Available: https://mermaid.js.org

[20] Anthropic, "Claude Code." 2025. Available: https://claude.ai/code

[21] Intel Corporation, "Avalon interface specifications," 2021.

[22] J. Bergeron, "Verification planning," in *Writing testbenches: Functional verification of HDL models*, Kluwer Academic Publishers, 2003.

A `can_mac_pcs_fce` Signal-Level Schematic

The schematic below shows the signal-level realization of the layered architecture described in Section 5, with typed record interfaces at every module boundary.

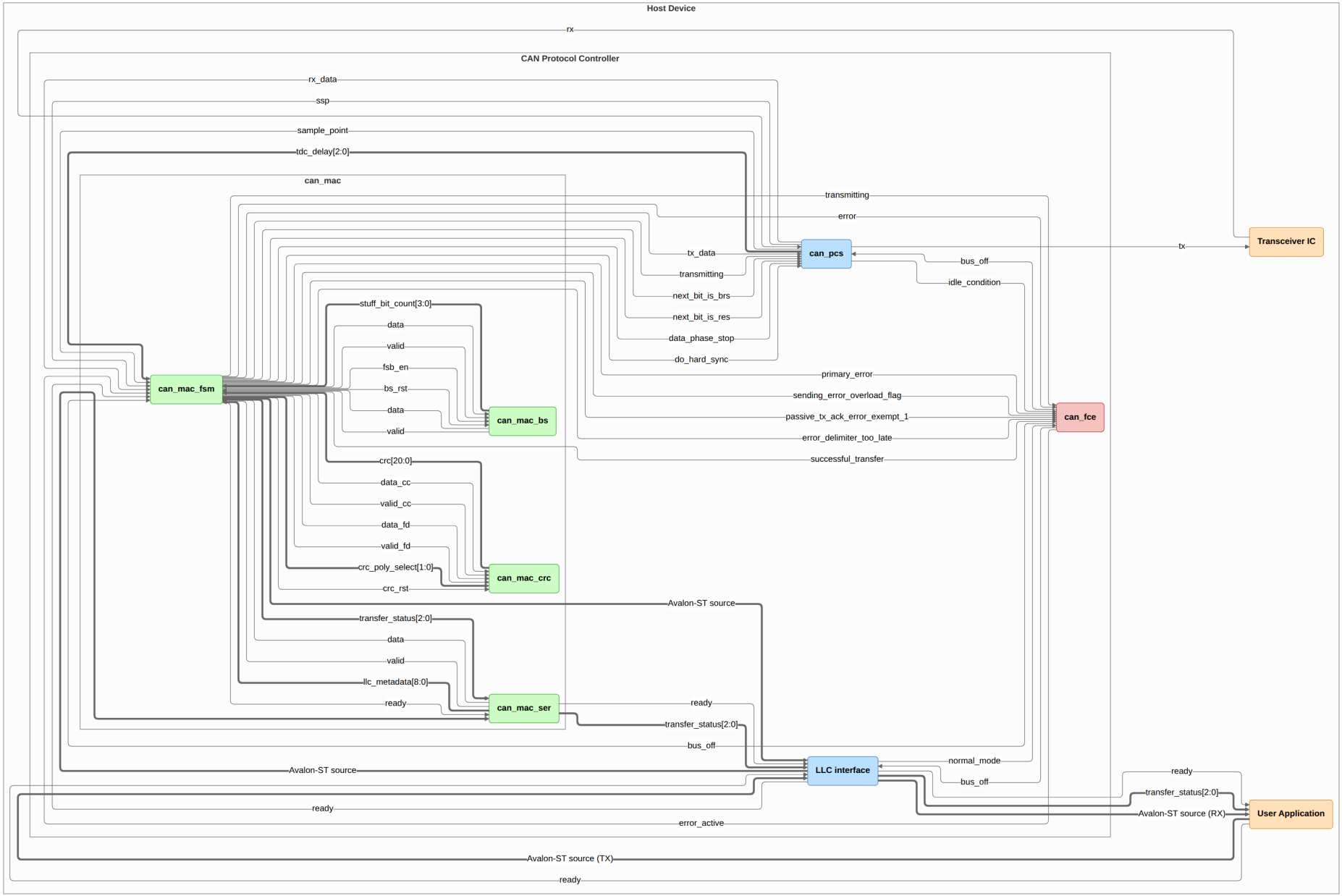

Figure 25: Signal-level schematic of `can_mac_pcs_fce`, showing the MAC, PCS, and FCE sub-layers and all inter-module interfaces. The MAC sub-layer is expanded to show its four internal entities (`can_mac_fsm`, `can_mac_ser`, `can_mac_bs`, `can_mac_crc`). The box labeled **LLC interface** is the stub for the unimplemented `can_llc` module.

B Verification Plan

Both tables are regenerated automatically from `verification_plan/verification_plan.toml` on each PDF build. The ID field is the join key between them. See Section 2.2 and Section 4 for the meaning of each field. The first table lists each requirement with its ISO source clause, priority, and paraphrase. The second table lists the verification metadata: layer, side, format applicability, observability, method, status, traceability label, file, and coverage criteria.

Table 10: ISO 11898-1 extracted requirements.

ID	ISO ref	Priority	Requirement	Notes
REQ-001	6.4.5.2	P1	**LLC notification** LLC shall notify the user on every successful DF/RF TX or RX.	Timestamping (§6.5.7) is optional per ISO and is out of scope.
REQ-002	6.4.5.5.2, 6.4.5.5.3	P2	**LLC transmission request and abort timing** 1) A TX request shall be processed no later than the second SOF after the request when no EFs are present. 2) Frames not yet passed to MAC are aborted immediately. Frames already in MAC are only aborted on error or arbitration loss - the abort request is held pending until then. 3) An abort request shall be processed before the second SOF (no EFs present) or third SOF (with EFs present).	
REQ-003	6.5.2, 6.5.3	P1	**Retransmission** 1) Frames losing arbitration shall be retransmitted unless aborted by the LLC user.	
REQ-004	6.5.4	P3	**Frame acceptance filtering** 1) Receivers should use acceptance filtering to decide frame relevance. 2) Filtering should be a single self-contained operation with no state carry-over from previous frames.	
REQ-005	6.5.6	P1	**LLC transfer status reporting** LLC shall report five transfer status values: Ongoing, Transmitted, Lost_Arbitration, Disturbed, and Aborted, each reflecting the corresponding MAC sub-layer event.	
REQ-006	6.6.4.4	P1	**CRC rules** 1) CRC_15 for CC frames. CRC_17 for FD frames with payload ≤16 B. CRC_21 for FD frames with payload >16 B. 2) Init vector: all-zero for CRC_15. MSB-one for CRC_17 and CRC_21.	

ID	ISO ref	Priority	Requirement	Notes
REQ-007	6.6.5.2, 6.6.5.3, 6.6.6.2, 6.6.6.3	P1	**Error and overload flags** 1) Active error and overload flags: 6 consecutive dominant bits each. 2) Passive error flag: 6 consecutive recessive bits (may be overwritten by dominant bits from other nodes). 3) Error and overload delimiters: 8 recessive bits each. 4) After the flag, all nodes monitor the bus and simultaneously begin the remaining 7-bit delimiter on detecting the first recessive bit.	
REQ-008	6.6.7.1, 6.6.7.2, 6.6.7.3, 6.6.7.4, Figure 6, Figure 7	P1	**Inter-frame space** 1) Intermission shall be exactly 3 recessive bits. 2) No DF or RF may be started during intermission. 3) EFs and OFs shall not be preceded by inter-frame space. 4) Bus idle is recognised at the third recessive intermission bit (error-active nodes/receivers), the eighth recessive suspend bit (error-passive transmitters), or on leaving bus-integration state. 5) An error-passive transmitter shall suspend for 8 bit times after intermission. If another node transmits during the suspend window, the node shall become a receiver.	
REQ-009	6.6.7.5	P1	**Bus re-integration** 1) Nodes shall start bus re-integration on protocol startup or bus-off recovery. 2) Re-integration completes after 11 consecutive recessive bits at the sample point. 3) For FD-capable nodes, any synchronisation-causing edge shall reset the 11-bit recessive-bit counter.	
REQ-010	6.6.7.3, 6.6.8, Figure 10, Figure 11	P1	**Frame initiation** 1) A node shall send SOF only when the bus is idle. 2) A dominant third intermission bit causes a node with a pending frame (if error-active or previous receiver) to skip SOF and begin arbitration immediately. 3) A dominant bit detected during bus idle shall be interpreted as SOF.	

ID	ISO ref	Priority	Requirement	Notes
REQ-011	6.6.10.1, 6.6.10.4, Figure 10, Figure 11, Figure 12, Figure 13	P2	**Remote frame** 1) RTR bit shall be dominant in data frames and recessive in remote frames. 2) Remote frames shall have no data field. 3) RF DLC indicates the requested data length of the corresponding DF.	
REQ-012	6.6.10.2, 6.6.11.2, Figure 12, Figure 13, Figure 18, Figure 19	P1	**SRR and RRS reserved bits** 1) Receivers shall accept recessive and dominant SRR bits without form error. 2) Receivers shall accept recessive and dominant RRS bits without form error.	
REQ-013	6.6.10.5, 6.6.11.5	P1	**CRC bit stream coverage** 1) CC: SOF, arbitration, control, and data fields (dynamic stuff bits excluded). 2) FD: SOF, arbitration, control, data, dynamic stuff bits, and stuff count field (fixed stuff bits excluded).	
REQ-014	6.6.10.6, 6.6.11.6, 6.6.14, Figure 17	P1	**Frame acknowledgement** 1) Receivers shall ACK consistent frames. 2) FD frames tolerate up to a 2-bit dominant ACK overlap to compensate for receiver phase differences. 3) ACK shall not depend on the frame identifier.	
REQ-015	6.6.11.3, Figure 20, Figure 21	P2	**ESI bit transmission** 1) Recessive when the LLC ESI flag is set. 2) Dominant when LLC ESI flag is clear and the node is error-active. 3) Recessive when LLC ESI flag is clear and the node is error-passive.	
REQ-016	6.6.11.5, Table 8	P1	**Stuff bit count (SBC) field** 1) SBC: 3-bit Gray-coded count of dynamic stuff bits (0-7) plus parity bit, placed before the CRC field. 2) TX shall transmit its dynamic stuff-bit count as the SBC field. 3) RX shall verify the received SBC against its own count.	

ID	ISO ref	Priority	Requirement	Notes
REQ-017	6.6.10.5, 6.6.11.5, Figure 16, Figure 22, Figure 23	P1	**CRC delimiter** 1) CC: CRC delimiter is one recessive bit. 2) FD: transmitter sends one recessive bit but shall accept a second recessive bit before the ACK slot.	
REQ-018	6.6.13.1, 6.6.13.2, 6.6.13.3.1, Table 10	P1	**Bit stuffing** 1) After 5 consecutive identical bits, insert a complementary dynamic stuff bit. RX discards it. 2) CC: dynamic stuffing spans SOF to FCRC. FD: SOF to end of data field. ACK and EOF are not stuffed. 3) FD CRC field: a fixed stuff bit precedes the SBC field and follows every fourth CRC bit, each the inverse of the preceding bit. No double stuffing.	
REQ-019	6.6.10.7, 6.6.11.7, 6.6.15.2, Figure 8, Figure 9	P1	**End of frame (EOF)** 1) A receiver shall validate a frame after the sixth EOF bit. 2) A transmitter requires all seven EOF bits to be error-free.	
REQ-020	6.6.10.2, 6.6.11.2, 6.6.17.4, 6.6.17.5, Figure 10, Figure 11, Figure 12, Figure 13, Figure 18, Figure 19	P1	**Bus arbitration** Recessive-sent, dominant-monitored: lose arbitration and become a receiver.	

ID	ISO ref	Priority	Requirement	Notes
REQ-021	6.6.21.2	P1	**MAC error detection** 1) Bit error: sent/monitored mismatch. Exceptions: recessive non-stuff bit during arbitration, dominant bit while sending a passive error flag. 2) Stuff error: sixth consecutive identical bit in a dynamically-stuffed field. 3) Form error: unexpected fixed-form bit value, or FSB not inverse of preceding bit (FB/FE). Exception: dominant last EOF bit or last delimiter bit is not a form error. 4) CRC error: CRC mismatch. FD also requires SBC match and correct parity. 5) ACK error: transmitter detects no dominant bit in the ACK slot.	Sub-claim 1 (bit error) is simulation-verified via recessive injection in test_bus_off. Sub-claims 2-5 require frame-aware error injection not available in the current testbench. All four are covered by code inspection only.
REQ-022	6.6.21.3.1, Figure 28, Figure 29, Figure 30, Figure 31	P1	**Error flag initiation and CRC error behaviour** 1) On any detected error, start an EF at the next bit and inform the LLC. 2) Data-phase errors: switch to nominal bit rate before starting the EF. 3) CRC error: receiver withholds ACK and sends an EF at the first EOF bit.	
REQ-023	6.6.21.3.2	P1	**Overload frame** 1) LLC-requested OF: Starts at the first intermission bit, at most 2 per inter-frame space. 2) Reactive OF: triggered by a dominant bit at the first two intermission bits, the last EOF bit (receivers only), or the last error/overload delimiter bit. Starts one bit after the trigger.	
REQ-024	7.3.2, 7.3.3, Table 13, Figure 32	P1	**PCS bit timing configuration** 1) TQ = m × one clock period, where m is a programmable integer prescaler. 2) A bit time consists of four segments: Sync_Seg (1 TQ), Prop_Seg, Phase_Seg1, and Phase_Seg2. The bus sample point falls at the end of Phase_Seg1. 3) IPT (Information Processing Time): the time a node requires to determine the bit value after sampling. IPT ≤ 2 TQ. 4) Separate nominal and FD data bit rate configurations, each based on a programmable integer prescaler.	Deployment constraints on the system, not RTL behaviors: prop_seg, SJW, Phase_Seg2, and oscillator tolerance must be programmed to meet the bounds in §7.3.2, §7.3.3, and §7.3.6. Minimum configuration ranges are given in Table 13 (§7.3.3).

ID	ISO ref	Priority	Requirement	Notes
REQ-025	7.3.4, 6.6.21.3.1, Figure 34, Figure 35	P1	**Transmitter delay compensation** 1) TDC applies in the CF data phase when BRS is recessive. Prescaler must be 1 or 2. 2) SSP position = measured transceiver loop delay + configured offset. 3) TDC active: bit errors at the SP are ignored. SSP errors are flagged at the following SP.	
REQ-026	7.3.5.1, 7.3.5.2, 7.3.5.3, 7.3.5.4, Figure 33	P1	**PCS synchronisation** 1) Phase error: the time interval between an R-to-D edge and the Sync_Seg boundary of the current bit time. Positive when the edge falls after Sync_Seg (late), negative when before (early). 2) Qualifying edge: R-to-D, previous SP is R. 3) SJW is the maximum per-resynchronization adjustment to Phase_Seg1 or Phase_Seg2. Qualifying edges cause resynchronization. If |phase error| ≤ SJW: same effect as hard synchronization. If phase error > SJW: Phase_Seg1 += SJW. If phase error < -SJW: Phase_Seg2 -= SJW. 4) One synchronization per bit time, re-enabled after the next R SP. 5) Hard synchronization: IFS edges (except first intermission bit), bus-integration state, and FDF-to-res transition in FD frames. Restarts bit time with Sync_Seg completed.	RTL is stricter than ISO: sync is suppressed unconditionally when transmitting. Safe since the transmitter is the timing source.
REQ-027	6.6.7.5, 7.4.2.2, 7.4.2.3, 8.1.4.4	P1	**PCS bus-off isolation and signalling** 1) During bus-off, the PCS shall not drive dominant bits. 2) PCS enters bus-off on a supervisor request and exits on a release request.	

ID	ISO ref	Priority	Requirement	Notes
REQ-028	8.1.4.2 rule a), rule b), rule c), rule d), rule e), rule f), rule g), rule h)	P1	**Error counter rules** 1) REC +1 on any receiver-detected error (except bit errors during error/overload flag). 2) REC +8: first dominant bit after sending an error flag, or bit error while sending an active error/overload flag. 3) REC -1 on successful RX if 1-127. Stays 0 if already 0. Set to 119-127 if above 127. 4) TEC +8 when sending an error flag. Error-passive ACK error is exempt. 5) TEC +8 on bit error while sending an active error or overload flag. 6) TEC -1 on successful TX (floor 0). 7) After 14 consecutive dominant bits post active error/overload flag (8 post passive error flag): both +8. Each additional 8 dominant bits: both +8.	
REQ-029	8.1.3.1, 8.1.3.2, 8.1.4.3, 8.1.4.4, Figure 43	P1	**Error state transitions** 1) Error-passive: TEC or REC > 127. 2) Error-active: TEC and REC ≤ 127. 3) Bus-off: TEC > 255. 4) Bus-off recovery: 128 idle conditions. TEC and REC reset to zero. 5) FCE notifies LLC on bus-off entry. 6) LLC resets FCE to initial state on request.	Idle condition: 128 × 11 recessive bits monitored
REQ-030	6.6.11.3, Figure 20, Figure 21	P1	**BRS bit rate switching** 1) Recessive BRS bit: switch to CF data rate at the BRS sample point. Dominant BRS: keep nominal rate. 2) Rate switches back to nominal at the first CRC delimiter sample point.	
REQ-031	6.4.3, 6.4.4, Table 5	P1	**DLC byte-count mapping** 1) DLC 0-8: linear mapping (CC and CF). 2) CF: DLC 9⟶12, 10⟶16, 11⟶20, 12⟶24, 13⟶32, 14⟶48, 15⟶64 bytes. 3) CC: DLC 9-15 clamped to 8 bytes.	

ID	ISO ref	Priority	Requirement	Notes
REQ-032	6.6.16	P1	**Byte/Bit transmission order** Within the data field byte 0 is sent first. Within each byte bit(7) is sent first.	
REQ-033	8.1.5, Figure 7	P1	**Node start-up** 1) A single node receiving no ACK becomes error-passive but never bus-off.	Verified via FCE Exception 1 mechanism (passive_tx_ack_error_exempt_1) in can_fce_tb.
REQ-034	6.6.18	P3	**MAC data consistency** If MAC frames are stored in shared memory, data consistency shall be ensured by buffering the whole frame before transmission starts, or by LLC error-checking during transfer.	Conditional requirement: applies only if shared memory is used. This implementation streams frames directly from LLC to MAC with no shared memory. the LLC is responsible for buffering and retransmission. Not applicable to current architecture.
REQ-035	6.6.21.1, 6.6.21.4	P2	**Error signalling enable** A configuration parameter shall exist to enable or disable error signalling.	
REQ-036	6.4.4	P3	**DLC padding** An LLC restricted to fewer bytes shall replace excess bytes with 0xCC padding in both TX and RX directions.	Only applicable if the implementation exposes a configurable maximum-data-byte restriction. If not implemented, waive.

ID	ISO ref	Priority	Requirement	Notes
REQ-037	5.2, Figure 2	P1	**Frame field sequence** 1) CBFF DF: SOF (D) → Arbitration (ID[28:18], RTR (D)) → Control (IDE (D), r0 (D), DLC[3:0]) → Data → CRC (CRC_15, delimiter (R)) → ACK (slot, delimiter (R)) → EOF (7×R). 2) CBFF RF: as CBFF DF but RTR (R) and no data field. 3) CEFF DF: SOF (D) → Arbitration (ID[28:18], SRR (R), IDE (R), ID[17:0], RTR (D)) → Control (r1 (D), r0 (D), DLC[3:0]) → Data → CRC (CRC_15, delimiter (R)) → ACK (slot, delimiter (R)) → EOF (7×R). 4) CEFF RF: as CEFF DF but RTR (R) and no data field. 5) FBFF DF: SOF (D) → Arbitration (ID[28:18], RRS (D)) → Control (IDE (D), FDF (R), res (D), BRS, ESI, DLC[3:0]) → Data (FD rate) → CRC (SBC[3:0], CRC_17/21, delimiter (R)) → ACK (slot, delimiter (R)) → EOF (7×R). 6) FEFF DF: SOF (D) → Arbitration (ID[28:18], SRR (R), IDE (R), ID[17:0], RRS (D)) → Control (FDF (R), res (D), BRS, ESI, DLC[3:0]) → Data (FD rate) → CRC (SBC[3:0], CRC_17/21, delimiter (R)) → ACK (slot, delimiter (R)) → EOF (7×R).	Notation: (D) = dominant, (R) = recessive. Unannotated bits (ID, DLC, BRS, ESI, CRC value, SBC, ACK slot) are data-dependent or conditional.

Table 11: ISO 11898-1 verification plan.

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-001	LLC	both	CB, CE, FB, FE	black_box	simulation	not_started			
REQ-002	LLC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-003	LLC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-004	LLC	receiver	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-005	LLC	transmitter	CB, CE, FB, FE	black_box	simulation	not_started			
REQ-006	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	complete	crc_15, crc_17, crc_21	can_mac_crc_tb	Three bins must each be hit: (1) CB or CE frame (CRC_15), (2) FB or FE frame with data ≤ 16 B (DLC ≤ 10, CRC_17), (3) FB or FE frame with data > 16 B (DLC ≥ 11, CRC_21).
REQ-007	system	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~900, test_bus_off	can_mac_fsm, can_mac_pcs_fce_tb	
REQ-008	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~486, test_normal, test_bus_off	can_mac_fsm, can_mac_pcs_fce_tb	
REQ-009	system	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~433, test_bus_off	can_mac_fsm, can_mac_pcs_fce_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-010	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~451, test_normal	can_mac_fsm, can_mac_pcs_fce_tb	
REQ-011	MAC	both	CB, CE	white_box	simulation	complete	FTYP Coverage	can_mac_pcs_fce_tb	Two bins: FTYP=0 (data frame) and FTYP=1 (remote frame) must each be hit.
REQ-012	MAC	both	CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~546, p_fsm:~548, test_normal	can_mac_fsm, can_mac_pcs_fce_tb	
REQ-013	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~299, p_fsm:~347, p_fsm:~983, test_normal	can_mac_fsm, can_mac_pcs_fce_tb	
REQ-014	system	both	CB, CE, FB, FE	white_box	simulation, wave-form_inspection	complete	test_normal	can_mac_pcs_fce_tb	
REQ-015	MAC	transmitter	FB, FE	white_box	simulation, wave-form_inspection	complete	test_normal	can_mac_pcs_fce_tb	
REQ-016	MAC	both	FB, FE	black_box	simulation	complete	p_sbc_checker	can_mac_bs_tb	
REQ-017	MAC	transmitter	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~768, p_fsm:~796, test_normal	can_mac_fsm, can_mac_pcs_fce_tb	
REQ-018	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	complete	Input Coverage, Output Coverage	can_mac_bs_tb	All bins in Input Coverage and Output Coverage must be hit.
REQ-019	MAC	both	CB, CE, FB, FE	white_box	simulation	complete	test_normal	can_mac_pcs_fce_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-020	system	transmitter	CB, CE, FB, FE	black_box	simulation	complete	test_lost_arb	can_mac_pcs_fce_tb	
REQ-021	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	in_progress	p_fsm:~283, p_fsm:~336, p_fsm:~343, p_fsm:~415, test_bus_off	can_mac_fsm, can_mac_pcs_fce_tb	Sub-claims 2-5 require frame-aware error injection. One scenario each must be confirmed: stuff error (6 consecutive identical bits in a dynamically-stuffed field), form error (illegal fixed-form bit value), CRC error (corrupted CRC bit with receiver withholding ACK), ACK error (no receiver drives dominant in ACK slot).
REQ-022	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~283, p_fsm:~294, p_fsm:~357, test_bus_off	can_mac_fsm, can_mac_pcs_fce_tb	
REQ-023	MAC	both	CB, CE, FB, FE	white_box	code_inspection	complete	p_fsm:~370, p_fsm:~384, p_fsm:~944	can_mac_fsm	
REQ-024	PCS	both	CB, CE, FB, FE	white_box	simulation	complete	test_normal	can_pcs_tb	
REQ-025	PCS	transmitter	FB, FE	black_box	simulation	complete	p_check_tdc_delay	can_pcs_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-026	PCS	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_can_pcs:~177, p_can_pcs:~228, test_normal	can_pcs, can_pcs_tb	
REQ-027	PCS	both	CB, CE, FB, FE	white_box	simulation, code_inspection	complete	p_can_pcs:~280, p_can_pcs:~379, test_bus_off	can_pcs, can_pcs_tb, can_mac_pcs_fce_tb	
REQ-028	FCE	both	CB, CE, FB, FE	black_box	simulation	complete	test_rule_a, test_rule_b, test_rule_c, test_rule_c_exception, test_rule_d, test_rule_e, test_rule_f_tx, test_rule_f_rx, test_rule_g, test_rule_h	can_fce_tb	
REQ-029	FCE	both	CB, CE, FB, FE	black_box	simulation	complete	test_reset, test_bus_off, test_bus_off_recovery	can_fce_tb	
REQ-030	MAC	both	FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~636, p_fsm:~755, test_normal	can_mac_fsm, can_mac_pcs_fce_tb	
REQ-031	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	complete	dlc_cov; test_normal	can_mac_ser_tb.vhd; can_mac_pcs_fce_tb	All 16 DLC bins (0-15) must be hit.
REQ-032	MAC	both	CB, CE, FB, FE	black_box	simulation	complete	test_normal	can_mac_pcs_fce_tb	
REQ-033	system	both	CB, CE, FB, FE	black_box	simulation	complete	test_rule_c_exception; normal_test, test_bus_off (waveform)	can_mac_pcs_fce_tb	
REQ-034	MAC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-035	MAC	both	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-036	LLC	both	CB, CE, FB, FE	black_box	simulation	not_started			
REQ-037	MAC	both	CB, CE, FB, FE	white_box	waveform_inspection	complete	test_normal	can_mac_pcs_fce_tb	